

LAB 1: Sensor

Name

- นายกิตติพิชญ์ สงไข 68340500007
- นายจิรายุ เรือนงาม 68340500013
- นางสาวพริยา กันสิงห์ 68340500042

Objectives

- เพื่อศึกษากระบวนการวัดค่าทางวิศวกรรม และหลักการทำงานของเซนเซอร์ประเภทต่าง ๆ ตั้งแต่การรับสัญญาณ ภายภาพไปจนถึงการแปลงเป็นหน่วยวัดทางฟิสิกส์
- เพื่อฝึกการอ่าน สุ่มตัวอย่าง และประมวลผลสัญญาณ จากเซนเซอร์ผ่านการเขียนโปรแกรมควบคุมบอร์ด ไมโครคอนโทรลเลอร์ (บอร์ด Nucleo)
- เพื่อศึกษาและพัฒนาทักษะการเทียบมาตรฐาน (Calibration) รวมทั้งการประมวลผลสัญญาณเพื่อลดสัญญาณรบกวน (Noise Reduction) เพื่อเพิ่มความแม่นยำของข้อมูล
- เพื่อประยุกต์ใช้กระบวนการทางวิทยาศาสตร์ ตั้งแต่การตั้งสมมติฐาน ออกแบบการทดลอง ดำเนินการทดลอง ไปจนถึงการเขียนรายงานเชิงวิชาการสรุปผลการทดลอง

1.1 Lab Potentiometer

การทดลองที่ 1 Linear Potentiometer

จุดประสงค์

1. เพื่อศึกษาคุณสมบัติและหลักการทำงานของ Potentiometer ที่เปลี่ยนการเคลื่อนที่เชิงเส้น (Linear Movement) ให้เป็นสัญญาณไฟฟ้า
2. เพื่อทดสอบ หาคุณสมบัติความเป็นเชิงเส้น และเปรียบเทียบการตอบสนองของ Linear Potentiometer กับ ข้อมูลทางทฤษฎีใน Datasheet

สมมติฐาน

- กราฟ Linear Potentiometer Type A จะมีความสัมพันธ์กับแรงดันไฟฟ้าเป็นกราฟเส้นโค้งแบบไม่เป็นเชิงเส้น (Non-linear)
- กราฟ Linear Potentiometer Type B จะมีความสัมพันธ์กับแรงดันไฟฟ้าเป็นกราฟเส้นตรง (Linear) ตลอดช่วงการเคลื่อน

ตัวแปร

1. ตัวแปรต้น:
 - ระยะการเคลื่อนของ Linear Potentiometer (cm)
2. ตัวแปรตาม:
 - ค่า ADC (Analog-to-Digital Converter) ที่อ่านได้ ตั้งแต่ 0-4095
 - แรงดันไฟฟ้า Output (V)
3. ตัวแปรควบคุม:
 - แรงดันไฟฟ้า Input $V_{CC} = 3.3 V$
 - บอร์ดประมวลผล Nucleo STM32G474RE

เอกสารและงานวิจัยที่เกี่ยวข้อง

Linear Potentiometer ใช้หลักการของ Voltage divider ซึ่งได้แรงดันไฟฟ้าขาออกมาตามสมการ

$$V_{out} = \frac{R_2}{R_1 + R_2} \cdot V_{in}$$

Potentiometer ประกอบด้วย แถบความต้านทาน, หน้าสัมผัสแบบเลื่อน (Wiper) ที่เคลื่อนที่ไปตามแถบความต้านทานและแนบสัมผัสทางไฟฟ้าเข้ากับส่วนใดส่วนหนึ่งของแถบ, ขั้วไฟฟ้าที่ปลายแต่ละด้านของแถบความต้านทาน, กลไกสำหรับเลื่อนหน้าสัมผัสจากปลายด้านหนึ่งไปยังอีกด้านหนึ่ง และตัวเรือน ที่บรรจุแถบความต้านทานและหน้าสัมผัสเอาไว้ภายใน ซึ่งการเลื่อน Wiper ไปยังจุดต่าง ๆ จะเป็นการแบ่งแถบความต้านทานเป็น 2 ค่าคือ ค่า R_1 และ R_2 ส่งผลให้แรงดันไฟฟ้าที่ออกจากขั้ว Wiper เปลี่ยนแปลงไปตามตำแหน่งนั้น ๆ

การตอบสนองต่อแรงดัน (Taper)

Logarithmic/Audio Taper (Type A): ค่าความต้านทานจะเปลี่ยนแปลงแบบเส้นโค้งโดยไม่ได้แปรผันตรงกับ การหมุน ทำให้ความต้านทานจะเปลี่ยนแปลงช้าในช่วงแรก และเร็วขึ้นอย่างมากในช่วงท้าย ซึ่งการออกแบบกราฟให้โค้งแบบนี้ทำขึ้นเพื่อให้สอดคล้องกับธรรมชาติการรับรู้ความดังของมนุษย์ จึงถูกนำไปใช้กับปุ่มปรับระดับเสียงในเครื่องเสียงเป็นหลัก เพื่อให้ผู้ใช้งานรู้สึกว่ายางง่าย ๆ ดังขึ้นอย่างเป็นธรรมชาติ

Linear Taper (Type B): ค่าความต้านทานจะเปลี่ยนแปลงในอัตราที่คงที่และแปรผันตรงกับระยะการหมุน อย่างสม่ำเสมอโดยมีกราฟความสัมพันธ์เป็นเส้นตรง รูปแบบนี้เหมาะสำหรับการตั้งค่าหรือการควบคุมที่ต้องการความแม่นยำและต้องการให้ผลลัพธ์เปลี่ยนไปตามสัดส่วนการหมุนแบบตรงไปตรงมา

การแปลงสัญญาณ Analog เป็น Digital (Analog-to-Digital Conversion) แรงดันไฟฟ้าที่ได้จาก Potentiometer เป็นสัญญาณ Analog ที่มีค่าต่อเนื่อง แต่ไมโครคอนโทรลเลอร์จะประมวลผลข้อมูลในรูปแบบ Digital จึงจำเป็นต้องมีการแปลงสัญญาณก่อนโดยใช้วงจร ADC (Analog-to-Digital Converter) ในการทดลองนี้ เราใช้บอร์ด Nucleo STM32G474RE ซึ่งมี ADC ที่มีความละเอียด 12 bit หมายความว่า มันสามารถแบ่งระดับแรงดัน Input (0-3.3V) ได้ละเอียดถึง 2^{12} หรือ 4096 ระดับ ทำให้ค่า Digital ที่อ่านได้จากโปรแกรมมีค่าอยู่ในช่วง 0 ถึง 4095

ขั้นตอนการดำเนินงาน

1. เชื่อมต่อบอร์ด Nucleo STM32G474RE เข้ากับ Linear Potentiometer (Type A และ B) โดยต่อขา 1 หรือ ขา 3 (ขาขอบ) เข้ากับ 3.3V และ GND และต่อขา 2 (ขากลาง) เข้ากับขา Analog Input ที่กำหนดไว้ใน Simulink
2. การทดลอง: เลื่อนแกนของ Linear Potentiometer จากตำแหน่งเริ่มต้น (0cm) ไปยังตำแหน่งสิ้นสุด (6cm) โดยหยุดเพื่อบันทึกค่าทุก ๆ 0.5 cm
3. การประมวลผลผ่าน MATLAB Script: ณ ตำแหน่งที่หยุด ให้รันโปรแกรมผ่าน MATLAB Script เพียงตัวเดียว โดย Script จะสั่งรัน Simulink Model เป็นเวลา 10 วินาที พร้อมทั้งดึงค่าสัญญาณดิจิทัล ADC (0-4095) มาแปลงเป็นค่าแรงดันไฟฟ้า (V_{out}) และคำนวณหาค่าเฉลี่ยอัตโนมัติ เพื่อลดความคลาดเคลื่อนจากสัญญาณรบกวน (Noise)

4. การประมวลผลข้อมูลและการบันทึกผล:

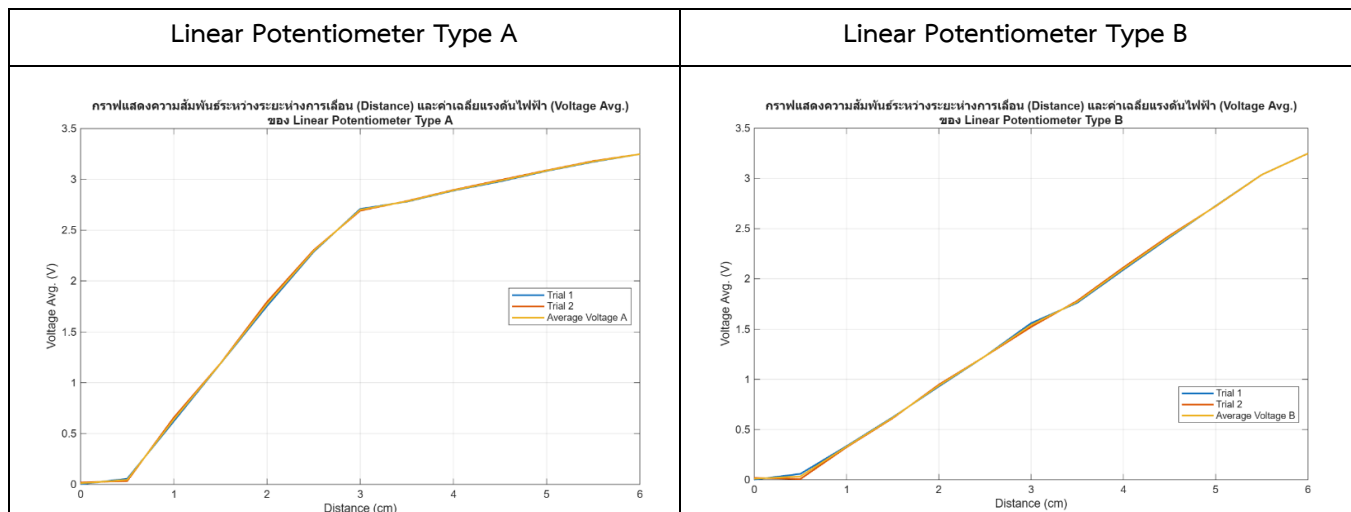
- แปลงค่าดิจิทัล ADC เป็นแรงดันไฟฟ้าเอาต์พุตผ่านบล็อก Gain โดยใช้สมการ ดังนี้

$$V_{out} = \left(\frac{ADC}{4095} \right) \times 3.3 \text{ v}$$

- บันทึกค่าแรงดันไฟฟ้าเฉลี่ยจากการทดลองทั้ง 3 ครั้งลงในโปรแกรม Excel เพื่อคำนวณหาค่าเฉลี่ยรวม (Voltage Average) ณ แต่ละตำแหน่ง
- นำข้อมูลแรงดันไฟฟ้าเฉลี่ย ณ แต่ละตำแหน่งไปสร้างกราฟเพื่อเปรียบเทียบแนวโน้ม (Trend) และความสม่ำเสมอ (Linearity) ของผลการทดลอง

ผลการทดลอง

ตารางที่ 1 การเปรียบเทียบความสัมพันธ์ระหว่างแรงดันไฟฟ้ากับระยะทางของ Linear Potentiometer Type A และ Type B



สรุปผลการทดลอง

จากการทดลองพบว่า ผลลัพธ์ที่ได้มีความสอดคล้องกับสมมติฐานที่ตั้งไว้ โดย Linear Potentiometer Type A ให้ผลลัพธ์การแปลงค่าแรงดันไฟฟ้า (V_{out}) เป็นเส้นโค้ง ซึ่งแสดงถึงคุณสมบัติแบบไม่เป็นเชิงเส้น (Non-linear) ในขณะที่ Linear Potentiometer Type B แสดงความสัมพันธ์ระหว่างระยะทางกับแรงดันไฟฟ้า (V_{out}) เป็นเส้นตรงอย่างชัดเจน ตลอดช่วงการเลื่อน 0-6 cm ทุก ๆ ระยะ 0.5 cm ซึ่งแสดงถึงคุณสมบัติความเป็นเชิงเส้น (Linearity)

นอกจากนี้ ผลการเก็บข้อมูลอัตโนมัติเป็นเวลา 10 วินาที ผ่านการประมวลผลด้วย MATLAB Script และคำนวณหาค่าเฉลี่ยรวมจากการทดสอบซ้ำ 3 ครั้ง มีความสอดคล้องกันสูงมากและมีความคลาดเคลื่อน (Error) จากสัญญาณรบกวนต่ำ แสดงถึงความเสถียรและความน่าเชื่อถือของระบบการวัด

อภิปรายผล

กราฟเส้นโค้งของ Type A (รุ่น PTA6043-2015DPA103) แสดงให้เห็นถึงคุณสมบัติ Audio Taper หรือ Logarithmic ได้อย่างชัดเจน การที่แรงดันไฟฟ้าเปลี่ยนแปลงช้าในช่วงแรกและเพิ่มขึ้นอย่างรวดเร็วในช่วงท้ายนั้น เป็นคุณสมบัติที่ถูกออกแบบมาโดยเฉพาะเพื่อให้สอดคล้องกับการรับรู้เสียงของมนุษย์ การทดลองนี้สามารถแสดงให้เห็นถึงคุณสมบัติการตอบสนองแบบไม่เป็นเชิงเส้นดังกล่าวได้เป็นอย่างดี

กราฟเส้นตรงของ Type B (รุ่น PTA6043-2015DPB103) ยืนยันคุณสมบัติ Linear Taper ได้เป็นอย่างดี ซึ่งเกิดจากการที่แถบความต้านทานภายในมีคุณสมบัติตลอดความยาวสม่ำเสมอ ทำให้แรงดันไฟฟ้าที่ได้เปลี่ยนแปลงในอัตราส่วนที่ตามระยะทาง คุณสมบัตินี้ทำให้มันเหมาะอย่างยิ่งสำหรับงานที่ต้องการวัดตำแหน่งหรือระยะทางอย่างแม่นยำ

ความสอดคล้องกันของข้อมูลในการทดสอบซ้ำผ่าน MATLAB Script และ Simulink ณ ทุก ๆ ระยะ 0.5 cm (เส้นกราฟทั้ง 3 ครั้งทับซ้อนกัน) แสดงให้เห็นว่ากระบวนการทดลองมีความน่าเชื่อถือสูง และการใช้เทคนิคหาค่าเฉลี่ยจากการรันโปรแกรมเป็นเวลา 10 วินาที ณ แต่ละจุด สามารถลดผลกระทบของสัญญาณรบกวน (Noise) ได้อย่างมีประสิทธิภาพ

ข้อเสนอแนะ

- การปรับระยะด้วยมือและสังเกตด้วยสายตาทำให้เกิดความคลาดเคลื่อน (Human Error) และได้ค่าไม่สม่ำเสมอในตำแหน่งเดียวกัน หากต้องการข้อมูลแรงดันไฟฟ้าที่แม่นยำและละเอียดขึ้น ควรติดตั้ง Potentiometer เข้ากับกลไกควบคุมการเคลื่อนที่ความละเอียดสูง
- ควรประยุกต์ใช้ตัวกรองสัญญาณดิจิทัล (Digital Filter) บนบอร์ด STM32 โดยตรง เพื่อลดสัญญาณรบกวน (Noise) และเพิ่มความเสถียรในการอ่านค่าแบบ Real-time

อ้างอิง

- <https://randomnerdtutorials.com/electronics-basics-how-a-potentiometer-works/>
- https://www.digikey.co.th/th/articles/the-complete-guide-to-potentiometers?srsltid=AU7gw4XRA-7C9Abu8Jlgn-Urqw6YeZiRV_XFRazM7Cp1vUAZiR2dzye7

การทดลองที่ 2 Rotary Potentiometer และ Schmitt Trigger

จุดประสงค์

1. เพื่อศึกษาและเปรียบเทียบคุณลักษณะการตอบสนอง (Taper) ของ Rotary Potentiometer ทั้ง 3 ชนิด คือ Type A, Type B และ Type C
2. เพื่อทดลองและสังเกตการณ์ทำงานของวงจร Schmitt Trigger ในการแปลงสัญญาณ Analog เป็น Digital

สมมติฐาน

- Rotary Potentiometer Type A จะมีความสัมพันธ์กับแรงดันไฟฟ้าเป็นกราฟเส้นโค้งแบบ Logarithmic
- Rotary Potentiometer Type B จะมีความสัมพันธ์กับแรงดันไฟฟ้าเป็นกราฟเส้นตรง (Linear)
- Rotary Potentiometer Type C จะมีความสัมพันธ์กับแรงดันไฟฟ้าเป็นกราฟเส้นโค้งแบบ Inverse Logarithmic
- วงจร Schmitt Trigger จะเปลี่ยนสถานะ Output High/Low ณ จุดแรงดันที่กำหนดไว้อย่างชัดเจน

ตัวแปร

1. ตัวแปรต้น:
 - มุมการหมุนของ Rotary Potentiometer (%)
2. ตัวแปรตาม:
 - ค่า ADC (Analog-to-Digital Converter) ที่อ่านได้ ตั้งแต่ 0-4095
 - สถานะ Output ของ Schmitt Trigger (Logic 1[3.3V] หรือ 0[0V])
 - แรงดันไฟฟ้า Output (V)
3. ตัวแปรควบคุม:
 - แรงดันไฟฟ้า Input $V_{CC} = 3.3\text{ V}$
 - บอร์ดประมวลผล Nucleo STM32G474RE

เอกสารและงานวิจัยที่เกี่ยวข้อง

Linear Potentiometer ใช้หลักการของ Voltage divider ซึ่งได้แรงดันไฟฟ้าขาออกมาตามสมการ

$$V_{out} = \frac{R_2}{R_1 + R_2} \cdot V_{in}$$

Potentiometer ประกอบด้วย แถบความต้านทาน, หน้าสัมผัสแบบเลื่อน (Wiper) ที่เคลื่อนที่ไปตามแถบความต้านทานและแนบสัมผัสทางไฟฟ้าเข้ากับส่วนใดส่วนหนึ่งของแถบ, ขั้วไฟฟ้าที่ปลายแต่ละด้านของแถบความต้านทาน, กลไกสำหรับเลื่อนหน้าสัมผัสจากปลายด้านหนึ่งไปยังอีกด้านหนึ่ง และตัวเรือน ที่บรรจุแถบความต้านทานและหน้าสัมผัส

เอาไว้ภายใน ซึ่งการเลื่อน Wiper ไปยังจุดต่าง ๆ จะเป็นการแบ่งแถบความต้านทานเป็น 2 ค่าคือ ค่า R_1 และ R_2 ส่งผลให้แรงดันไฟฟ้าที่ออกจากขั้ว Wiper เปลี่ยนแปลงไปตามตำแหน่งนั้น ๆ

คุณลักษณะการตอบสนองของแรงดัน (Taper)

Logarithmic/Audio Taper (Type A): ค่าความต้านทานจะเปลี่ยนแปลงแบบเส้นโค้งโดยไม่ได้แปรผันตรงกับ การหมุน ทำให้ความต้านทานจะเปลี่ยนแปลงช้าในช่วงแรก และเร็วขึ้นอย่างมากในช่วงท้าย ซึ่งการออกแบบกราฟให้โค้งแบบนี้ทำขึ้นเพื่อให้สอดคล้องกับธรรมชาติการรับรู้ความดังของมนุษย์ จึงถูกนำไปใช้กับปุ่มปรับระดับเสียงในเครื่องเสียงเป็นหลัก เพื่อให้ผู้ใช้สามารถรู้สึกละเอียดง่าย ๆ ดังขึ้นอย่างเป็นธรรมชาติ

Linear Taper (Type B): ค่าความต้านทานจะเปลี่ยนแปลงในอัตราที่คงที่และแปรผันตรงกับระยะการหมุน อย่างสม่ำเสมอโดยมีกราฟความสัมพันธ์เป็นเส้นตรง รูปแบบนี้เหมาะสำหรับการตั้งค่าหรือการควบคุมที่ต้องการความแม่นยำและต้องการให้ผลลัพธ์เปลี่ยนไปตามสัดส่วนการหมุนแบบตรงไปตรงมา

Reverse Audio Taper (Type C): ค่าความต้านทานจะเปลี่ยนแปลงอย่างรวดเร็วในช่วงแรกที่เริ่มหมุน และค่อย ๆ เปลี่ยนแปลงช้าลงในช่วงท้าย (ทำงานตรงข้ามกับ Type A) มักใช้ในวงจรปรับแต่งความถี่เฉพาะทาง หรือใช้เป็นปุ่มควบคุมเสียงในเครื่องดนตรีสำหรับคนถนัดซ้าย

วงจร Schmitt Trigger เป็นวงจรเปรียบเทียบแรงดันที่ใช้หลักการป้อนกลับแบบบวก เพื่อแปลงสัญญาณอนาล็อกที่มีการแกว่งตัวให้เป็นสัญญาณดิจิทัลรูปคลื่นสี่เหลี่ยมที่เสถียร วงจรนี้ช่วยแก้ปัญหาเอาต์พุตสลับสถานะไปมา ผิดปกติจากสัญญาณ noise ด้วยการใช้อนุสมบัตินิสเทรีซิส (Hysteresis) ซึ่งหมายถึงการมีจุดเปลี่ยนสถานะแยกเป็น 2 ระดับ ได้แก่ ขีดจำกัดแรงดันระดับสูง สำหรับสัญญาณขาขึ้น และขีดจำกัดแรงดันระดับต่ำ สำหรับสัญญาณขาลง ผลต่างของแรงดันทั้งสองจะทำหน้าที่ป้องกันสัญญาณรบกวน ป้องกันไม่ให้วงจรสลับสถานะโดยไม่ตั้งใจ ทำให้ได้สัญญาณดิจิทัลที่สะอาดและแม่นยำ

การแปลงสัญญาณ Analog เป็น Digital (Analog-to-Digital Conversion) แรงดันไฟฟ้าที่ได้จาก Potentiometer เป็นสัญญาณ Analog ที่มีค่าต่อเนื่อง แต่ไมโครคอนโทรลเลอร์จะประมวลผลข้อมูลในรูปแบบ Digital จึงจำเป็นต้องมีการแปลงสัญญาณก่อนโดยใช้วงจร ADC (Analog-to-Digital Converter) ในการทดลองนี้ เราใช้บอร์ด Nucleo STM32G474RE ซึ่งมี ADC ที่มีความละเอียด 12 bit หมายความว่า มันสามารถแบ่งระดับแรงดัน Input (0-3.3V) ได้ละเอียดถึง 2^{12} หรือ 4096 ระดับ ทำให้ค่า Digital ที่อ่านได้จากโปรแกรมมีค่าอยู่ในช่วง 0 ถึง 4095

ขั้นตอนการดำเนินงาน

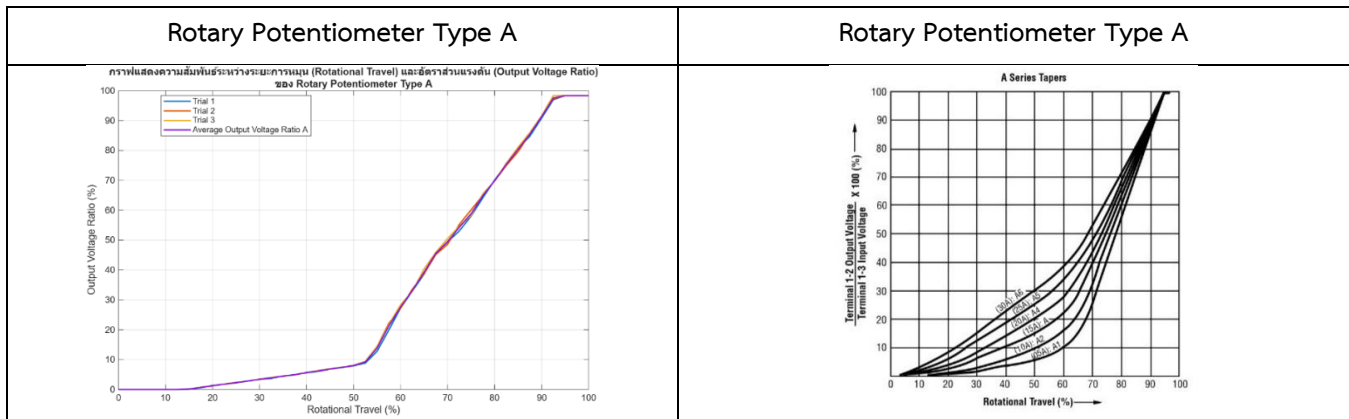
1. เชื่อมต่อบอร์ด Nucleo STM32G474RE เข้ากับ Rotary Potentiometer (Type A, B และ C) โดยต่อขา 1 หรือ ขา 3 (ขาขอบ) เข้ากับ 3.3V และ GND และต่อขา 2 (ขากลาง) เข้ากับขา Analog Input ที่กำหนดไว้ใน Simulink
2. การทดลอง: หมุนแกนของ Rotary Potentiometer จากตำแหน่ง 0% ไปยังตำแหน่ง 100% โดยจะหยุดเพื่อ บันทึกค่าตาม Scale ที่กำหนด ทุก ๆ 2.5%
3. การประมวลผลผ่าน MATLAB Script: ณ ตำแหน่งที่หยุด ให้รันโปรแกรมผ่าน MATLAB Script เพียงตัวเดียว โดย Script จะสั่งรัน Simulink Model เป็นเวลา 10 วินาที พร้อมทั้งดึงค่าสัญญาณดิจิทัล ADC (0-4095) มาแปลงเป็นค่าแรงดันไฟฟ้า (V_{out}) และคำนวณหาค่าเฉลี่ยอัตโนมัติ เพื่อลดความคลาดเคลื่อนจากสัญญาณรบกวน
4. การตั้งค่า Schmitt Trigger: ใน Simulink ได้มีการกำหนดค่าบล็อก Schmitt Trigger ในส่วน Inputs โดยตั้งค่า Low level input voltage ไว้ที่ 1.0V และ High level input voltage ไว้ที่ 2.5V เพื่อใช้เป็นเกณฑ์ตัดสินใจในการเปลี่ยนสถานะการทำงานของสัญญาณ
5. การประมวลผลข้อมูลและการบันทึกผล:
 - แปลงค่าดิจิทัล ADC เป็นแรงดันไฟฟ้าเอาต์พุตผ่านบล็อก Gain โดยใช้สมการ ดังนี้

$$V_{out} = \left(\frac{ADC}{4095} \right) \times 3.3 \text{ v}$$

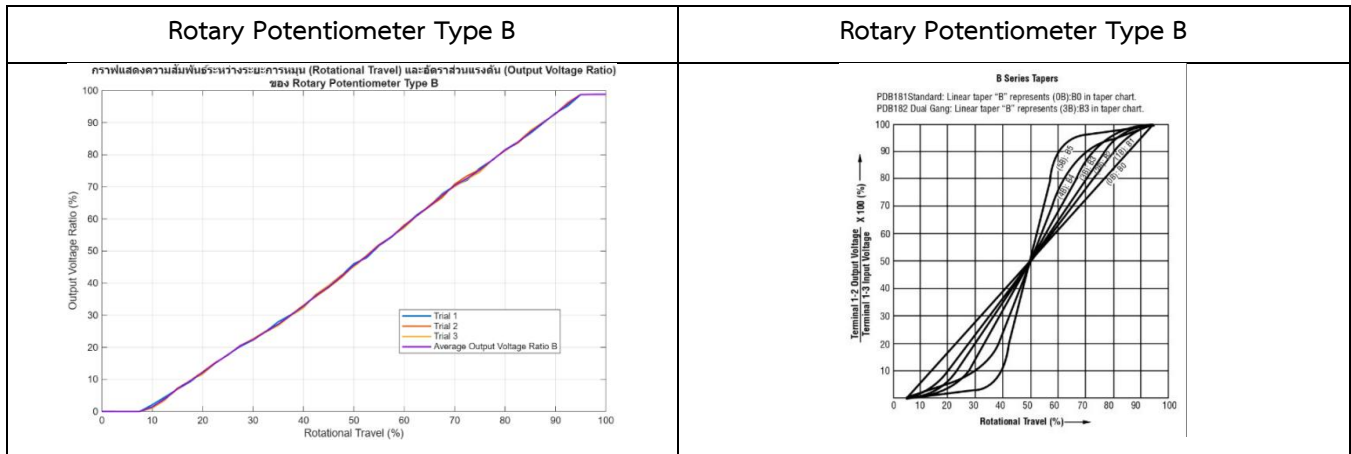
- บันทึกค่าแรงดันไฟฟ้าเฉลี่ยจากการทดลองทั้ง 3 ครั้งลงในโปรแกรม Excel เพื่อคำนวณหาค่าเฉลี่ยรวม (Voltage Average) ณ แต่ละตำแหน่ง
- นำข้อมูลแรงดันไฟฟ้าเฉลี่ย ณ แต่ละตำแหน่งไปสร้างกราฟเพื่อเปรียบเทียบแนวโน้ม (Trend) และความสม่ำเสมอ (Linearity) ของผลการทดลอง

ผลการทดลอง

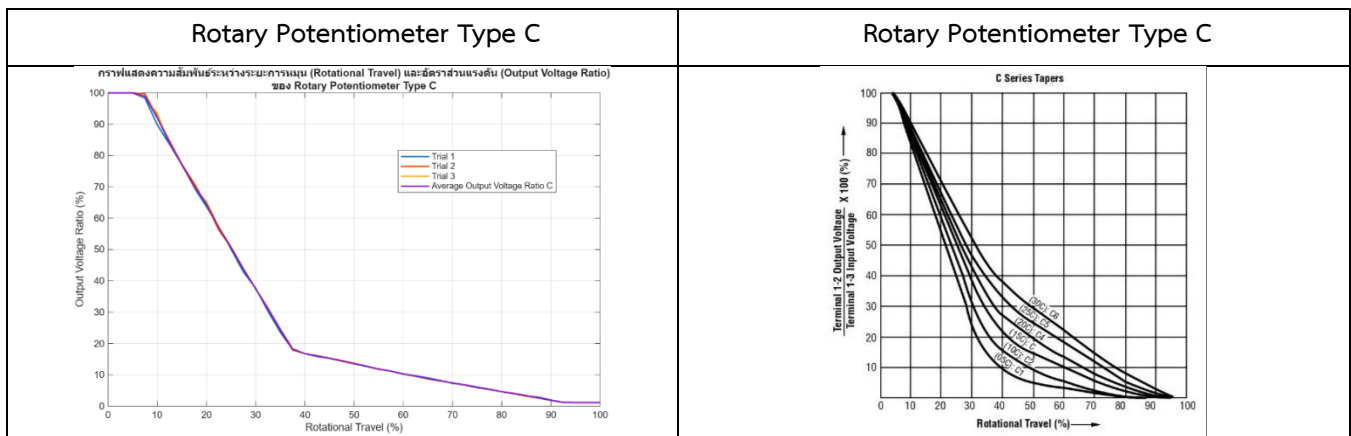
ตารางที่ 2 กราฟแสดงความสัมพันธ์ระหว่างระยะการหมุน และอัตราส่วนแรงดันไฟฟ้า ของ Rotary Potentiometer Type A จากการทดลอง 3 ครั้ง ค่าเฉลี่ยรวม และการเปรียบเทียบเส้นคุณลักษณะ Tapers จาก Datasheet



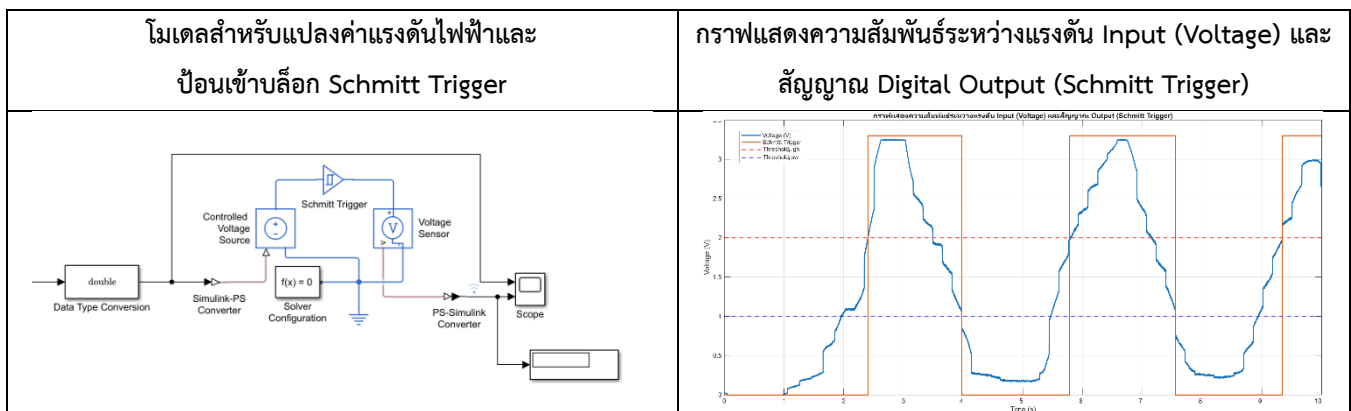
ตารางที่ 3 กราฟแสดงความสัมพันธ์ระหว่างระยะการหมุน และอัตราส่วนแรงดันไฟฟ้า ของ Rotary Potentiometer Type B จากการทดลอง 3 ครั้ง ค่าเฉลี่ยรวม และการเปรียบเทียบเส้นคุณลักษณะ Tapers จาก Datasheet



ตารางที่ 4 กราฟแสดงความสัมพันธ์ระหว่างระยะการหมุน และอัตราส่วนแรงดันไฟฟ้า ของ Rotary Potentiometer Type C จากการทดลอง 3 ครั้ง ค่าเฉลี่ยรวม และการเปรียบเทียบเส้นคุณลักษณะ Tapers จาก Datasheet



ตารางที่ 5 แสดงโมเดล Simulink สำหรับการประมวลผลสัญญาณแรงดันไฟฟ้าและป้อนเข้าบล็อก Schmitt Trigger (ภาพซ้าย) ร่วมกับกราฟเปรียบเทียบความสัมพันธ์ระหว่างแรงดันไฟฟ้าขาเข้า (Input Voltage) และสัญญาณดิจิทัลขาออก (Digital Output) (ภาพขวา)



สรุปผลการทดลอง

จากการทดลอง พบว่าผลลัพธ์เป็นไปตามสมมติฐานที่ตั้งไว้ทั้งหมด โดย Rotary Potentiometer Type A ให้กราฟเป็นเส้นโค้งแบบ Logarithmic, Type B ให้กราฟความสัมพันธ์ระหว่างมุมการหมุนและแรงดันไฟฟ้าเป็นเส้นตรง ซึ่งแสดงถึงคุณสมบัติความเป็นเชิงเส้น (Linearity), และ Type C ให้กราฟเป็นเส้นโค้งแบบ Inverse Logarithmic

ในส่วนของวงจร Schmitt Trigger สามารถทำหน้าที่แปลงสัญญาณ Analog (Voltage) ให้เป็นสัญญาณ Digital (High/Low) ได้สำเร็จ โดยสัญญาณ Output มีการเปลี่ยนแปลงตามสัญญาณ Input แบบ Real-time ณ จุดแรงดันไฟฟ้าที่กำหนดไว้คือ Upper Threshold ที่ 2.5V และ Lower Threshold ที่ 1.0V

อภิปรายผล

การเปรียบเทียบคุณสมบัติ Taper: เมื่อนำกราฟผลการทดลอง (ตารางที่ 2, 3 และ 4) มาเปรียบเทียบกับกราฟคุณลักษณะ Taper จาก Datasheet พบว่ามีความสอดคล้องกันอย่างยิ่ง

- Type A (รุ่น PDB181-K420K-103A2): กราฟเส้นโค้งที่มีความชันน้อยในช่วงแรกและชันขึ้นอย่างรวดเร็วในช่วงท้าย สอดคล้องกับคุณสมบัติ Audio Taper ที่ออกแบบมาให้เข้ากับการรับรู้เสียงของมนุษย์
- Type B (รุ่น PDB181-K420K-103B): กราฟเส้นตรงยืนยันคุณสมบัติ Linear Taper ได้อย่างสมบูรณ์ ซึ่งเหมาะสำหรับงานที่ต้องการการตอบสนองที่สม่ำเสมอ
- Type C (รุ่น PDB181-K420K-103C): กราฟที่มีลักษณะตรงกันข้ามกับ Type A ยืนยันคุณสมบัติ Inverse Audio Taper ได้อย่างชัดเจน

ความสอดคล้องกันของข้อมูลจากการทดลองทั้ง 3 ครั้งในแต่ละกราฟ แสดงให้เห็นว่ากระบวนการทดลองมีความน่าเชื่อถือสูง และการใช้เทคนิคหาค่าเฉลี่ยสามารถลดผลกระทบของสัญญาณรบกวนได้อย่างมีประสิทธิภาพ

การออกแบบและวิเคราะห์ Schmitt Trigger: ในการทดลองนี้ ได้ใช้บล็อก Schmitt Trigger ใน Simulink เพื่อจำลองการทำงานของวงจร โดยกำหนดค่า Upper Threshold ไว้ที่ 2.5V และ Lower Threshold ไว้ที่ 1.0V เหตุผลที่กำหนดค่าเช่นนี้ เพื่อสร้างช่วง ฮิสเทอรีซิส (Hysteresis Band) ขนาด 1.5V สำหรับป้องกันการสั่นของสถานะ (Chattering) จากสัญญาณรบกวน (Noise)

ผลลัพธ์ในตารางที่ 5 แสดงให้เห็นว่าสัญญาณ Digital เปลี่ยนสถานะเป็น HIGH เมื่อแรงดัน Input เพิ่มขึ้นผ่าน 2.5V และเปลี่ยนกลับเป็น LOW เมื่อแรงดัน Input ลดต่ำลงผ่าน 1.0V พอดี ซึ่งสอดคล้องกับการตั้งค่าทุกประการ และช่วยเพิ่มเสถียรภาพให้กับสัญญาณ Output ได้อย่างมีประสิทธิภาพตามทฤษฎีวงจร Schmitt Trigger

ข้อเสนอแนะ

- การปรับปรุงความแม่นยำในการวัดตำแหน่ง เพื่อลดความคลาดเคลื่อนจากการกำหนดตำแหน่งด้วยมือ (Human Error) ในการทดลองครั้งถัดไปควรใช้กลไกควบคุมการเคลื่อนที่ที่มีความแม่นยำสูง เช่น Stepper Motor สำหรับควบคุมการหมุน
- การศึกษาความกว้างของช่วง Hysteresis Band พิจารณาปรับเปลี่ยนระยะห่างระหว่าง Upper และ Lower Threshold (เช่น ขยายช่วงจากเดิม 1.5V เป็น 2.0V หรือแคบลงเป็น 0.8V) เพื่อศึกษาผลกระทบของความกว้าง Hysteresis Band ต่อความไวในการตอบสนอง (Response Sensitivity) และระดับความคงทนต่อสัญญาณรบกวน (Noise Immunity) ในสภาวะแวดล้อมที่มี Noise ต่างกัน

อ้างอิง

- https://bourns.com/docs/technical-documents/technical-library/sensors-controls/technical-notes/Bourns_panel_controls_taper_technote.pdf?sfvrsn=ad60c966_38
- <https://randomnerdtutorials.com/electronics-basics-how-a-potentiometer-works/>
- <https://www.electronics-tutorials.ws/opamp/op-amp-comparator.html>

1.2 Lab Magnetic Sensor

การทดลองที่ 1 การเปรียบเทียบคุณลักษณะระหว่างขั้วเหนือ (North Pole) และขั้วใต้ (South Pole)

จุดประสงค์

1. เพื่อศึกษาหลักการทำงานของ Magnetic Sensor โดยอาศัยหลักการ Hall Effect
2. เพื่อวิเคราะห์และเปรียบเทียบการตอบสนองของเซ็นเซอร์ (Magnetic Response) ต่อสนามแม่เหล็ก ขั้วเหนือและขั้วใต้ ที่เปลี่ยนแปลงไป
3. เพื่อศึกษาและวิเคราะห์ผลของแผ่นป้องกันสนามแม่เหล็ก (Magnetic Shielding) ที่มีต่อการวัดค่า

สมมติฐาน

- ยิ่งแม่เหล็กเข้าใกล้เซ็นเซอร์มากเท่าใด ค่าแรงดันไฟฟ้าที่วัดได้จะยิ่งเปลี่ยนแปลงไปจากค่าปกติ (V_0) มากขึ้นเท่านั้น
- ขั้วแม่เหล็กเหนือและใต้จะทำให้แรงดันไฟฟ้า Output เปลี่ยนแปลงในทิศทางตรงกันข้าม
- แผ่นป้องกันสนามแม่เหล็ก (Magnetic Shielding) จะลดทอนความแรงของสนามแม่เหล็ก ทำให้ค่าแรงดันไฟฟ้าที่วัดได้มีการเปลี่ยนแปลงน้อยลง เมื่อเทียบกับกรณีที่ไม่มีแผ่นป้องกัน

ตัวแปร

1. ตัวแปรต้น:
 - ขั้วของแม่เหล็ก (ขั้วเหนือ - North / ขั้วใต้ - South)
 - ระยะห่างระหว่างแม่เหล็กกับเซ็นเซอร์
 - การมี/ไม่มีแผ่นป้องกันสนามแม่เหล็ก (Shielding)
2. ตัวแปรตาม:
 - ค่า ADC (Analog-to-Digital Converter) ที่อ่านได้ ตั้งแต่ 0-4095
 - แรงดันไฟฟ้า Output $V_{cc} = 5V$
 - ค่าความหนาแน่นสนามแม่เหล็ก (B) ที่คำนวณได้ ในหน่วย milli Tesla
3. ตัวแปรควบคุม:
 - แรงดันไฟฟ้า Input
 - บอร์ดประมวลผล Nucleo STM32G474RE
 - เซ็นเซอร์ DRV5055 Ratiometric Linear Hall Effect Magnetic Sensor

เอกสารและงานวิจัยที่เกี่ยวข้อง

หลักการ Hall Effect คือปรากฏการณ์ทางฟิสิกส์ที่เกิดขึ้นเมื่อมีกระแสไฟฟ้าไหลผ่านแผ่นตัวนำซึ่งอยู่ในสนามแม่เหล็ก จะเกิดความต่างศักย์ไฟฟ้า (เรียกว่า Hall Voltage) ขึ้นในทิศทางที่ตั้งฉากกับทั้งทิศทางของกระแสและสนามแม่เหล็ก เซ็นเซอร์ DRV5055 ที่ใช้ในการทดลองนี้ ทำงานโดยอาศัยหลักการดังกล่าว โดยเป็น Ratiometric Linear Hall Effect Sensor ซึ่งหมายความว่าขนาดของ Hall Voltage ที่เกิดขึ้นจะแปรผันเป็นเส้นตรง ตามความหนาแน่นของสนามแม่เหล็ก ทำให้สามารถวัดความเข้มของสนามแม่เหล็กได้จากการวัดแรงดันไฟฟ้าที่ Output ของเซ็นเซอร์

คุณลักษณะของสนามแม่เหล็กและการป้องกัน

ความหนาแน่นของสนามแม่เหล็ก (Magnetic Flux Density, B): คือค่าที่ใช้วัดความเข้มและทิศทางของสนามแม่เหล็ก ณ บริเวณใดบริเวณหนึ่ง มีหน่วยเป็น เทสลา (Tesla, T) หรือ มิลลิเทสลา (mT) ในการทดลองนี้ จะคำนวณค่า B โดยอ้างอิงจากแรงดันไฟฟ้าที่วัดได้

แผ่นป้องกันสนามแม่เหล็ก (Magnetic Shielding): คือการใช้วัสดุที่มีคุณสมบัติในการเบี่ยงเบนหรือสร้างเส้นทางใหม่ให้กับเส้นแรงแม่เหล็ก แทนที่จะปิดกั้นโดยตรง วัสดุเหล่านี้มีความสามารถในการซึมซับสนามแม่เหล็กสูง ทำให้เส้นแรงแม่เหล็กไหลผ่านตัวเองแทนที่จะผ่านบริเวณที่ต้องการป้องกัน ส่งผลให้ความหนาแน่นของสนามแม่เหล็กบริเวณนั้นลดลง

สมการ Magnetic Response ของเซ็นเซอร์ จากคู่มือการทดลอง ความสัมพันธ์ระหว่างแรงดันไฟฟ้า Output (V_{OUT}) และความหนาแน่นของสนามแม่เหล็ก (B) สามารถอธิบายได้ด้วยสมการ

$$V_{OUT} = V_Q + B \times (\text{Sensitivity}_{25^\circ\text{C}} \times (1 + S_{TC} \times (T_A - 25^\circ\text{C})))$$

ในการทดลองนี้ จะทำการวัดค่า V_{OUT} และจัดรูปสมการใหม่เพื่อคำนวณค่า B โดยสมมติการทดลองทำที่อุณหภูมิประมาณ 25°C เพื่อความสะดวกในการคำนวณ สามารถจัดรูปสมการใหม่ได้ดังนี้

$$B[\text{mT}] = \frac{(V_{out}[\text{mV}] - V_Q[\text{mV}])}{\text{Sensitivity}[\frac{\text{mV}}{\text{mT}}]}$$

คุณลักษณะพารามิเตอร์อ้างอิงจาก Datasheet (DRV5055)

- Quiescent Voltage (V_Q): แรงดัน Output ณ สภาวะไม่มีสนามแม่เหล็ก ($B = 0\text{mT}$) มีค่าเท่ากับครึ่งหนึ่งของแรงดันไฟเลี้ยง $V_Q = 0.5 \times V_{cc} = 0.5 \times 5\text{V} = 2.5\text{V}(2500\text{mV})$
- Sensitivity: อัตราความไวของเซ็นเซอร์ DRV5055A3 มีค่าเท่ากับ 2.5 mV/mT ณ อุณหภูมิ 25°C

ขั้นตอนการดำเนินงาน

1. การติดตั้งและตั้งค่าเบื้องต้น

- ติดตั้งบอร์ด Nucleo STM32-G474RE เข้ากับบอร์ดทดลองเซ็นเซอร์ DRV5055
- ต่อสายสัญญาณระหว่างเซ็นเซอร์และบอร์ด Nucleo ดังนี้: ขั้ว VCC , GND, และ OUT เข้ากับพอร์ต ADC

2. ขั้นตอนการเก็บข้อมูล

- ทดลองขั้วเหนือ (North Pole): ปรับระยะห่างแท่งแม่เหล็กจากเซ็นเซอร์ตั้งแต่ 1.1 cm ถึง 4.5 cm (หยุดบันทึกค่าทุก ๆ 0.1 cm)
- ณ ทุกๆ ระยะที่หยุด ให้รันโปรแกรมผ่าน MATLAB Script โดยระบบจะสั่งงาน Simulink Model เป็นเวลา 10 วินาที เพื่อดึงค่าดิจิทัล ADC (0-4095) และทำ Sample Averaging อัตโนมัติเพื่อลดสัญญาณรบกวน
- ทดลองขั้วใต้ (South Pole): สลับขั้วแม่เหล็กและทำตามขั้นตอนเดิมซ้ำอีกครั้ง
- ทำการทดลองซ้ำทั้งหมด จำนวน 3 ครั้ง เพื่อนำค่ามาหาค่าเฉลี่ยรวม

3. การประมวลผลข้อมูลและการบันทึกผล

- แปลงค่าดิจิทัล ADC เป็นแรงดันไฟฟ้าเอาต์พุต (V_{out}) ผ่าน บล็อก Gain โดยใช้สมการ ดังนี้

$$V_{out} = \left(\frac{ADC}{4095} \right) \times 3300 \text{ mV}$$

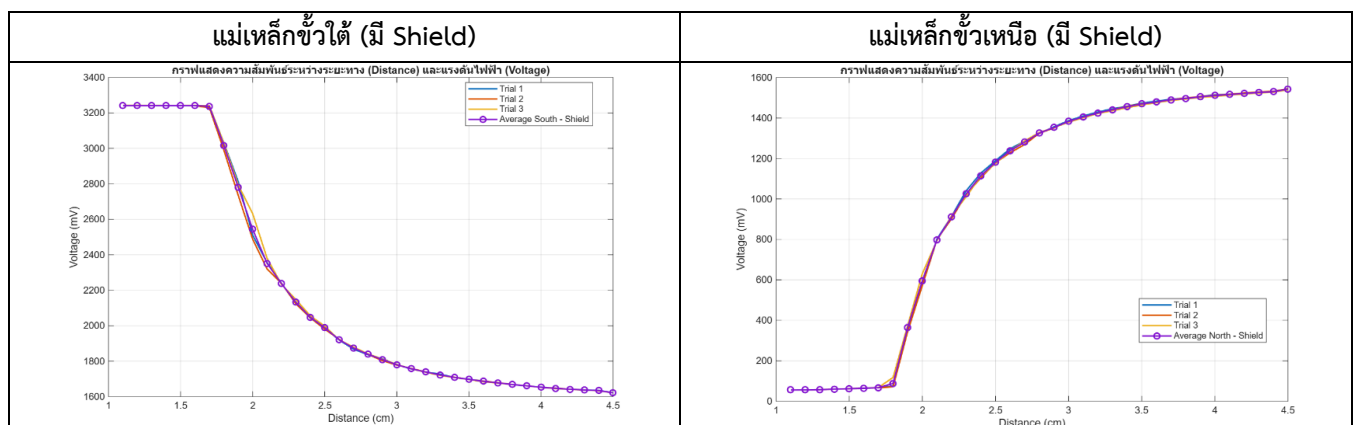
- ตัวอย่างขั้นตอนการคำนวณค่าความเข้มสนามแม่เหล็ก (B) เมื่อวัดค่าแรงดันไฟฟ้าได้ $V_{out} = 3125 \text{ mV}$ นำไปแทนค่าในสมการเพื่อหาค่าความเข้มสนามแม่เหล็ก (B) โดยใช้ค่า Quiescent Voltage ($V_Q = 2500 \text{ mV}$ ณ $V_{CC} = 5 \text{ V}$) และ Sensitivity = 2.5 mV/mT

$$B = \frac{(3125 - 2500)}{2.5}$$
$$B = 50 \text{ mT}$$

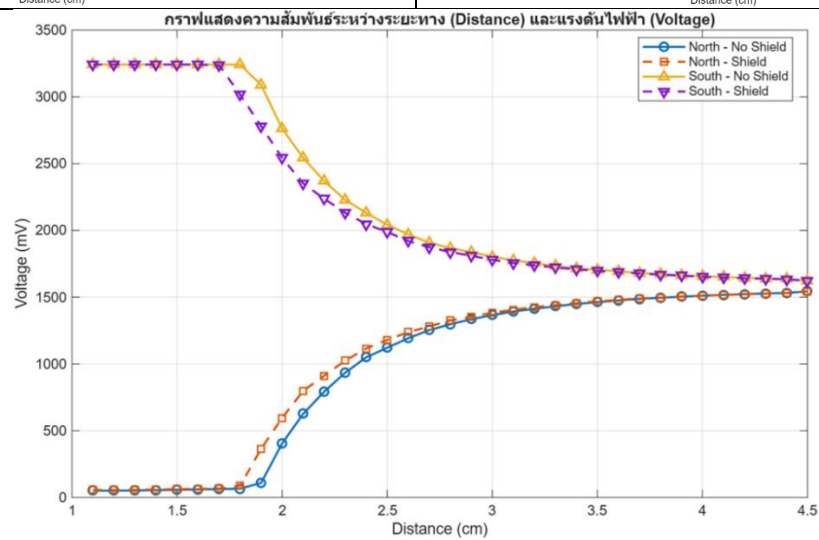
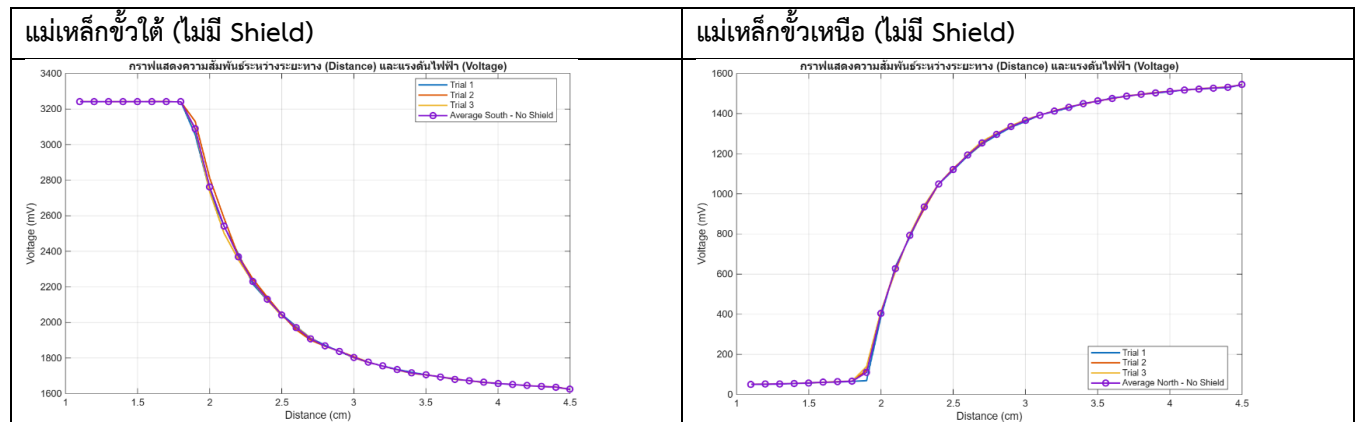
- บันทึกค่าแรงดันไฟฟ้าเฉลี่ยจากการทดลองทั้ง 3 ครั้งลงในโปรแกรม Excel เพื่อคำนวณหา ค่าเฉลี่ยรวม (Voltage Average) และคำนวณค่า ความหนาแน่นสนามแม่เหล็ก (B) ณ แต่ละตำแหน่ง
- นำข้อมูลแรงดันไฟฟ้าเฉลี่ยและค่าความหนาแน่นสนามแม่เหล็กไปพล็อตสร้างกราฟ เพื่อเปรียบเทียบแนวโน้ม (Trend) และความเป็นเชิงเส้น (Linearity) ของผลการทดลอง

ผลการทดลอง

ตารางที่ 6 การเปรียบเทียบความสัมพันธ์ระหว่างระยะการหมุนและแรงดันไฟฟ้าของขั้วเหนือและขั้วใต้ในสภาวะมีแผ่นกำบังสนามแม่เหล็ก (With Shield)

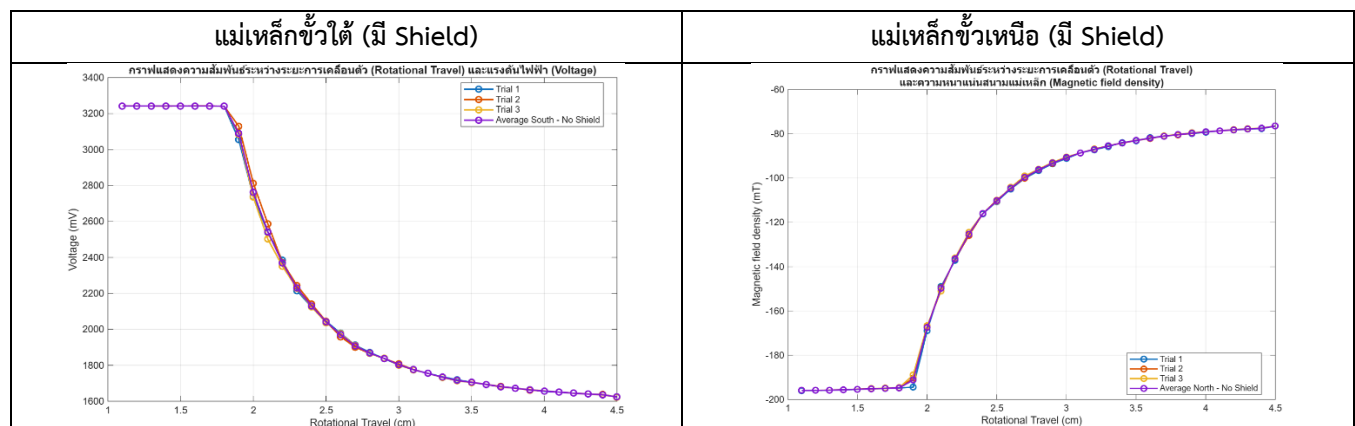


ตารางที่ 7 การเปรียบเทียบความสัมพันธ์ระหว่างระยะการหมุนและแรงดันไฟฟ้าของขั้วเหนือและขั้วใต้ในสภาวะไม่มีแผ่นกำบังสนามแม่เหล็ก (No Shield)

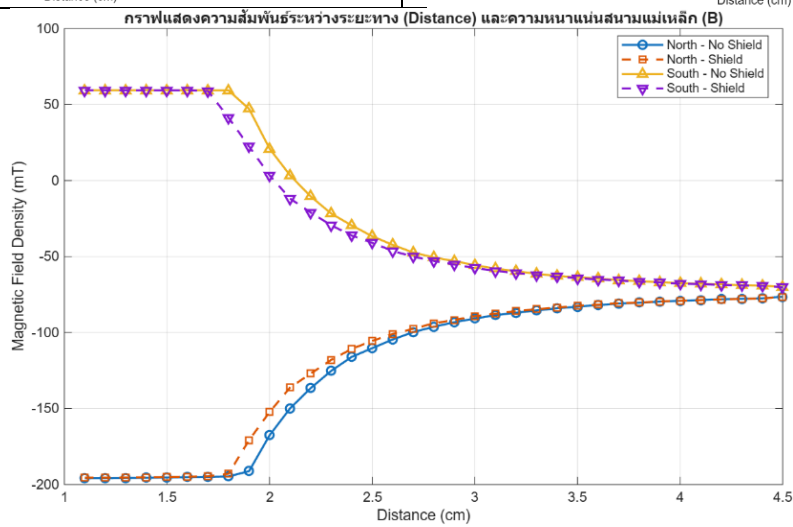
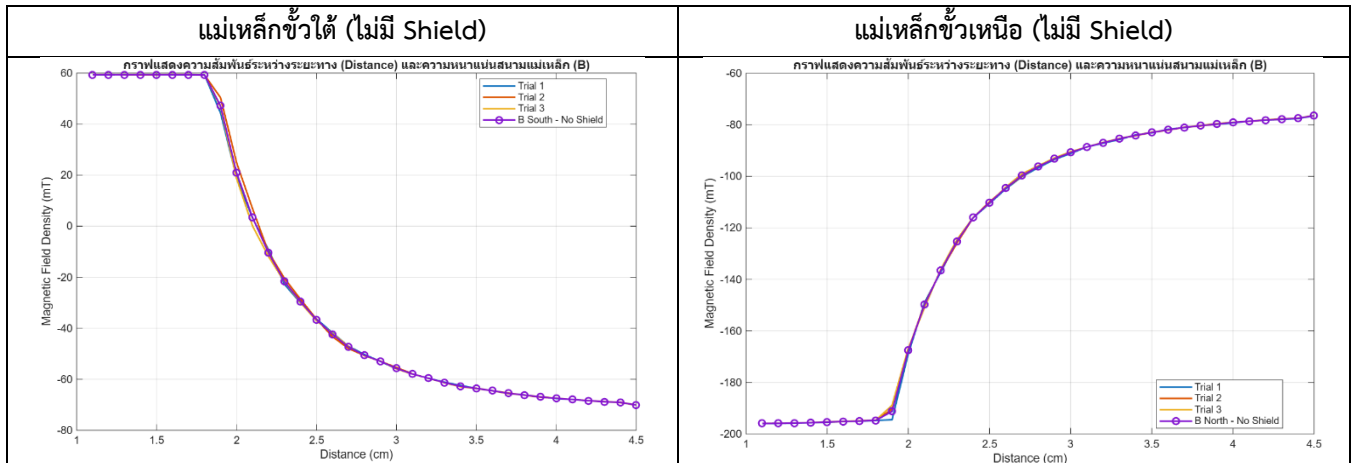


รูปที่ 1 ความสัมพันธ์ระหว่างระยะการหมุนและแรงดันไฟฟ้า

ตารางที่ 8 ความสัมพันธ์ระหว่างระยะทางและความหนาแน่นฟลักซ์แม่เหล็ก (B) ในสภาวะมีแผ่นกำบังสนามแม่เหล็ก

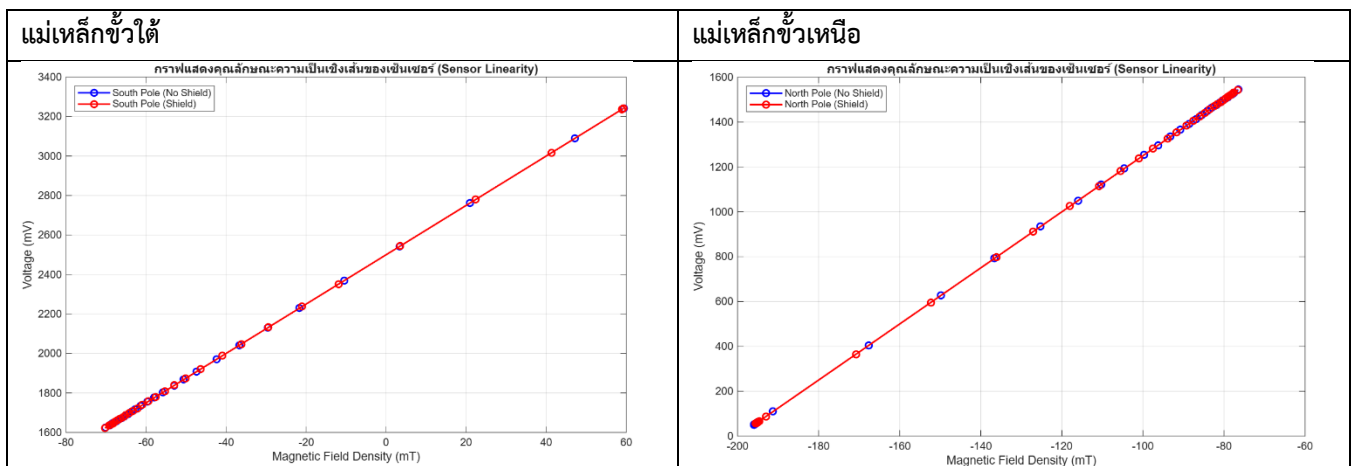


ตารางที่ 9 ความสัมพันธ์ระหว่างระยะห่างและความหนาแน่นฟลักซ์แม่เหล็ก (B) ในสถานะไม่มีแผ่นกำบังสนามแม่เหล็ก



รูปที่ 2 ความสัมพันธ์ระหว่างระยะการหมุนและความหนาแน่นฟลักซ์แม่เหล็ก (B)

ตารางที่ 10 คุณลักษณะความเป็นเชิงเส้นของเซนเซอร์ตรวจจับสนามแม่เหล็ก สัมพันธ์ระหว่างแรงดันไฟฟ้าเอาต์พุต (V) และความหนาแน่นฟลักซ์แม่เหล็ก (B)



จากการวิเคราะห์ความสัมพันธ์เชิงเส้น (Linear Regression) ของกราฟฝั่งซ้ายได้และซ้ายเหนือ พบว่ามีค่าสัมประสิทธิ์การตัดสินใจ $R^2 = 0.9998$ (หรือมีความคลาดเคลื่อน % Linearity Error < 0.2%) แสดงให้เห็นว่าเซนเซอร์ตอบสนองต่อค่าความเข้มสนามแม่เหล็กเป็นเส้นตรงอย่างสมบูรณ์ตลอดช่วงการวัด

สรุปผลการทดลอง

ผลการทดลองเป็นไปตามที่คาดการณ์ไว้ทั้งหมด โดยยิ่งแม่เหล็กเข้าใกล้เซ็นเซอร์ แรงดันไฟฟ้าจะยิ่งเกิดการเปลี่ยนแปลงมากขึ้น ดังนี้

1. การตอบสนองต่อขั้วแม่เหล็ก
 - แม่เหล็กขั้วใต้ ทำให้แรงดันไฟฟ้า เพิ่มขึ้น สูงกว่าค่า V_Q ขณะไม่มีสนามแม่เหล็ก
 - แม่เหล็กขั้วเหนือ ทำให้แรงดันไฟฟ้า ลดลง ต่ำกว่าค่า V_Q ขณะไม่มีสนามแม่เหล็ก
2. ผลของ Magnetic Shielding ทำให้เมื่อมี แผ่นป้องกันสนามแม่เหล็ก (Shield) สามารถลดทอนความเข้มของสนามแม่เหล็กที่เซ็นเซอร์ตรวจจับได้อย่างชัดเจน สังเกตได้จากกราฟแสดงการเปลี่ยนแปลงของแรงดันไฟฟ้าที่มีค่าเบี่ยงเบนจาก V_Q น้อยลงในกรณีที่มีแผ่น Shield ณ ระยะทางเดียวกัน

อภิปรายผล

1. หลักการทำงานและ Magnetic Response
 - ผลการทดลองยืนยันหลักการทำงานของ Hall Effect Sensor รุ่น DRV5055 ได้เป็นอย่างดี การที่ขั้วแม่เหล็กต่างกันให้การตอบสนองเชิงแรงดันในทิศทางตรงกันข้าม ถือเป็นไปตามคุณสมบัติเฉพาะของตัวเซ็นเซอร์
 - ลักษณะของกราฟที่เปลี่ยนแปลงตามระยะห่าง แสดงว่าสัญญาณ Output สามารถตอบสนองตามสัญญาณ Input แบบ Real-time จึงสามารถนำค่าแรงดันไฟฟ้าที่วัดได้ (Raw Signal) ไปคำนวณหาค่าความหนาแน่นสนามแม่เหล็ก (Magnetic Flux Density: B) ในหน่วย mT (มิลลิเทสลา) ได้อย่างแม่นยำ
2. ผลของ Magnetic Shielding
 - จากกราฟแสดงความสัมพันธ์ จะเห็นได้ว่าเส้นกราฟของกรณี มี Shield มีการเปลี่ยนแปลงของแรงดันไฟฟ้าน้อยกว่ากรณี ไม่มี Shield อย่างชัดเจน ซึ่งยืนยันว่าแผ่นป้องกันสามารถลดทอนความเข้มสนามแม่เหล็กได้จริง
 - แผ่น Shield ทำหน้าที่เบี่ยงเบนเส้นแรงแม่เหล็ก (Magnetic Flux Lines) ให้ไหลผ่านตัวแผ่น Shield เอง ส่งผลให้ความหนาแน่นสนามแม่เหล็กที่ผ่านไปยังตัวเซ็นเซอร์มีค่าน้อยลง

ข้อเสนอแนะ

- การปรับช่วงระยะทางการทดลอง ให้สามารถปรับและวัดค่าในระยะที่ใกล้กว่า 1.2cm ได้ เพื่อศึกษาพฤติกรรมและการตอบสนองของเซ็นเซอร์ในช่วงที่สัญญาณเริ่มอิ่มตัวให้มีความชัดเจนยิ่งขึ้น
- การศึกษาวัสดุแผ่น Shield ควรเพิ่มการทดลองเปรียบเทียบประสิทธิภาพกับวัสดุ Shielding ประเภทอื่น เช่น Mu-metal หรือ Aluminium เพื่อศึกษาพฤติกรรมการลดทอนสนามแม่เหล็กที่แตกต่างกัน

อ้างอิง

- <https://www.ti.com/lit/ds/symlink/drv5055.pdf>
- <https://www.electronics-tutorials.ws/electromagnetism/hall-effect.html>
- <https://www.allegromicro.com/en/insights-and-innovations/technical-documents/hall-effect-sensor-ic-publications/hall-effect-ic-applications-guide>

1.3 Lab Incremental Encoder

การทดลองที่ 1 การเปรียบเทียบประสิทธิภาพของ Incremental Encoder ในโหมด X1, X2 และ X4 ผ่านการวัดค่าพัลส์ องศาการหมุน และความเร็วรอบ

จุดประสงค์

1. ศึกษาหลักการทำงานของ Incremental Encoder และลักษณะของสัญญาณที่อ่านได้จาก Incremental Encoder
2. ศึกษาความแตกต่างของสัญญาณ X1 ,X2 และ X4 ของ Incremental Encoder
3. ศึกษาการแปลง Raw Counts เป็น Relative Position ,Angular Position และ Angular Velocity

สมมติฐาน

1. สัญญาณ X1 ,X2 และ X4 มีความแตกต่างกันแบบเชิงเส้น
2. สัญญาณ X4 มีความละเอียดของสัญญาณมากที่สุด
3. ความเร็วเชิงมุม (Angular Velocity) มีการเปลี่ยนแปลงตามระยะทาง Output (Angular Position)

ตัวแปร

1. ตัวแปรต้น:
 - ทิศทางการหมุน
 - ความเร็วการหมุน
2. ตัวแปรตาม:
 - สัญญาณของ Channel A และ B ของ Encoder Mode X1 ,X2 และ X4
 - ความเร็วเชิงมุม (Angular Velocity)
3. ตัวแปรควบคุม:
 - องศาเริ่มต้น-สิ้นสุดของการทดลอง
 - แรงดันไฟฟ้า $V_{CC} = 3.3 V$

เอกสารและงานวิจัยที่เกี่ยวข้อง

ลักษณะสัญญาณและหลักการทำงานของ Incremental Encoder แบบ 2 ช่องสัญญาณ มีหลักการทำงานโดยการสร้างสัญญาณเอาต์พุตออกมาในรูปแบบของคลื่นสี่เหลี่ยม 2 ช่อง ได้แก่ Channel A และ Channel B โดยสัญญาณทั้งสองจะมี Phase-shift ห่างกัน 90 องศา หรือที่เรียกว่าสัญญาณแบบควอดราเจอร์ การอ่านค่าตำแหน่งจะอาศัยวงจรนับทางอิเล็กทรอนิกส์ในการตรวจจับการเปลี่ยนสถานะของสัญญาณทั้งจากระดับต่ำไปสูงและสูงไปต่ำของทั้งสอง

ช่องสัญญาณ ซึ่งวิธีการนับลักษณะนี้ จะทำให้ระบบสามารถอ่านค่าตำแหน่งได้ถึง 4 ครั้งต่อ 1 รอบคลื่นไฟฟ้า ส่งผลให้ความละเอียดในการอ่านค่าเพิ่มขึ้นเป็น 4 เท่า นอกจากนี้ ใน Encoder บางรุ่นอาจมีการเพิ่มสัญญาณช่องที่ 3 หรือ สัญญาณ Index ซึ่งจะส่งพัลส์ออกมาเพียง 1 ครั้งต่อการหมุน 1 รอบ เพื่อใช้สำหรับระบุตำแหน่งอ้างอิงศูนย์

การถอดรหัสสัญญาณจาก Incremental Encoder สามารถแบ่งตามรูปแบบการนับขอบสัญญาณได้ 3 โหมดหลัก ซึ่งส่งผลโดยตรงต่อความละเอียดและภาระการประมวลผลของระบบ

เริ่มจาก โหมด X1 ระบบจะทำการนับการเปลี่ยนแปลงเพียงขอบเดียวของสัญญาณช่องใดช่องหนึ่งเท่านั้น เช่น ตรวจสอบเฉพาะขอบขาขึ้นของ Channel A ทำให้ความละเอียดของจำนวนพัลส์ต่อรอบเท่ากับสเปค PPR พื้นฐานของตัว Encoder โหมดนี้ใช้ทรัพยากรในการประมวลผลน้อยที่สุด โดยการระบุทิศทางจะประเมินจากสถานะลอจิกของ Channel B ในจังหวะที่เกิดขอบสัญญาณนั้น

ถัดมาคือ โหมด X2 (Double-Edge Decoding) ระบบจะเพิ่มความละเอียดขึ้นเป็นสองเท่าโดยการตรวจสอบทั้งขอบขาขึ้นและขอบขาลงของสัญญาณเพียงช่องเดียว ทำให้ระบบอ่านค่าจำนวนพัลส์ได้เป็น 2 เท่าของค่า PPR พื้นฐาน ถือเป็นการเพิ่มความละเอียดในการวัดตำแหน่งโดยที่บอร์ดประมวลผลยังไม่ต้องรับภาระหนักจากการอ่านสัญญาณทั้งสองช่องอย่างเต็มรูปแบบ

สุดท้ายคือ โหมด X4 (Quadrature Decoding) ซึ่งเป็นโหมดที่ให้ความละเอียดสูงสุด โดยระบบจะนับการเปลี่ยนแปลงของทั้งขอบขาขึ้นและขาลงจากทั้ง Channel A และ Channel B หรือ นับทุกครั้งที่มีการเปลี่ยนสถานะลอจิกเกิดขึ้นในระบบ ทำให้ความละเอียดถูกขยายสูงสุดถึง 4 เท่าของค่า PPR พื้นฐาน การระบุทิศทางในโหมดนี้จะใช้ตรรกศาสตร์ตรวจสอบสถานะของทั้งสองช่องร่วมกันในทุกเสี้ยวจังหวะ โหมด X4 จึงนิยมใช้ในงานที่ต้องการความละเอียดและความแม่นยำสูงมาก

การระบุทิศทางการหมุน ระบบสามารถประเมินทิศทางการหมุนของแกน Encoder ได้จากการวิเคราะห์ความสัมพันธ์ของเฟส (Phase relationship) ระหว่างคลื่นสัญญาณทั้งสองช่อง โดยหากแกนถูกหมุนไปในทิศทางหนึ่ง สัญญาณ Channel A จะเกิดการเปลี่ยนสถานะก่อนหน้าสัญญาณ Channel B แต่ในทางกลับกัน หากมีการหมุนแกนในทิศทางตรงกันข้าม สัญญาณ Channel B จะเปลี่ยนสถานะนำหน้าสัญญาณ Channel A ความแตกต่างของลำดับการเกิดสถานะนี้ ทำให้สามารถตรวจจับและแยกแยะทิศทางการหมุนของอุปกรณ์กลไกได้อย่างแม่นยำ

สมการทั่วไปเพื่อคำนวณความละเอียดเชิงมุม (Angular Resolution)

$$\text{Angular Resolution} = \frac{360^\circ}{PPR \times Multiplier}$$

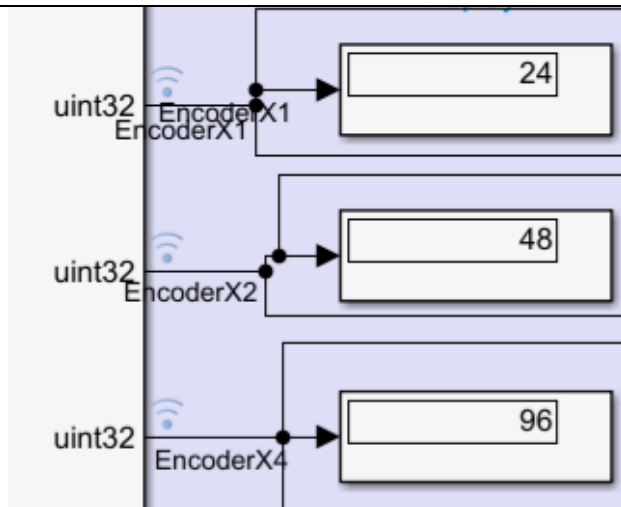
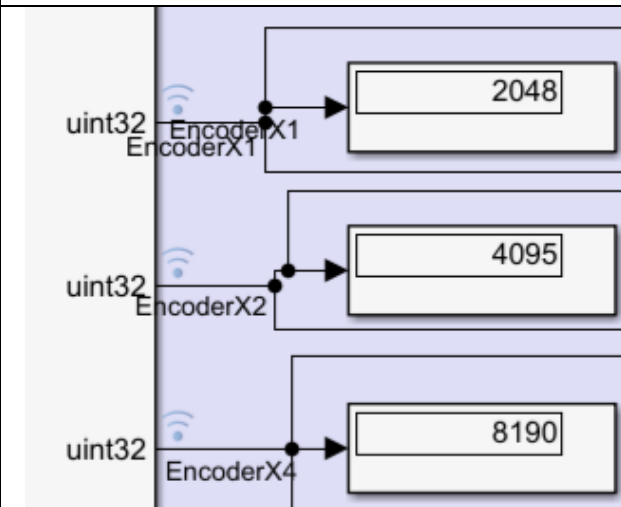
ขั้นตอนการดำเนินงาน

1. เชื่อมต่อบอร์ด Nucleo STM32G474RE เข้ากับ Incremental Encoder ขาไฟเลี้ยง เข้ากับแหล่งจ่ายไฟ 3.3V และ GND ส่วนขา A กับ B ต่อเข้ากับ ขา Digital Input ที่กำหนดไว้ใน manual
2. เขียน MATLAB function สำหรับการตัดค่าที่ overflow/underflow

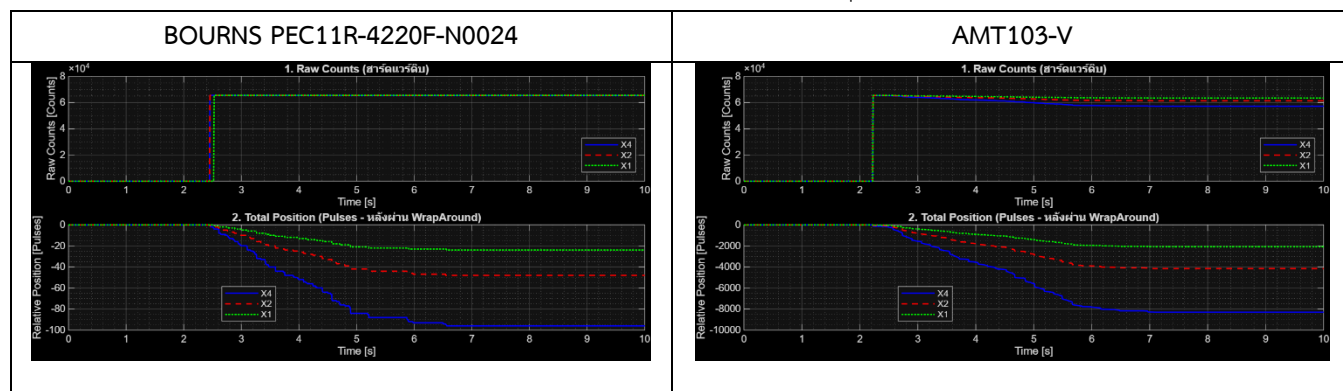
- หมุน encoder ให้ครบ 1 รอบเพื่อหา Pulses Per Revolution ของแต่ละโหมด และคำนวณหา Angular Resolution ซึ่งจะเป็น Gain สำหรับคำนวณเป็น angular position
- นำค่าที่ได้จาก gain ไปหาอนุพันธ์เพื่อจะได้เป็น angular velocity
- เก็บผลการทดลอง Encoder รุ่นละ 3 สถานการณ์ ใช้เวลาบันทึกผล 10 วินาทีต่อครั้ง โดยครั้งที่ 1 หมุนย้อนกลับ 1 รอบ เพื่อตรวจสอบการทำงานของฟังก์ชัน Wraparound ครั้งที่ 2 หมุนตามเข็มนาฬิกา 3 วินาที หยุดค้างไว้ 2 วินาที และหมุนทวนเข็มนาฬิกากลับมายังตำแหน่งเริ่มต้นครั้งที่ 3 หมุนแกนซ้าย อย่างสม่ำเสมอ บิดกระชากอย่างรวดเร็ว 1 จังหวะ แล้วกลับมาหมุนซ้ายๆ ต่อจนจบ
- นำผลที่ได้มาพล็อตกราฟ
- ทำซ้ำกับ encoder อีกรุ่น

ผลการทดลอง

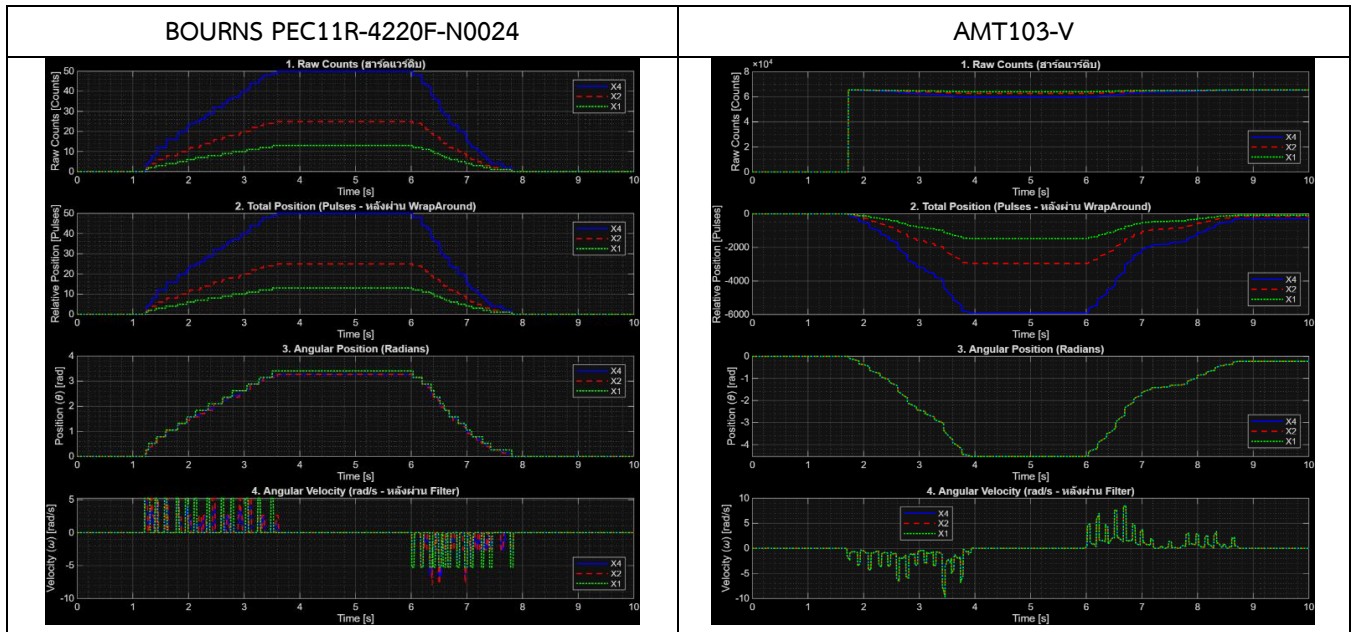
ตารางที่ 11 แสดงค่า counts ของแต่ละ Encoder

BOURNS PEC11R-4220F-N0024	AMT103-V
	

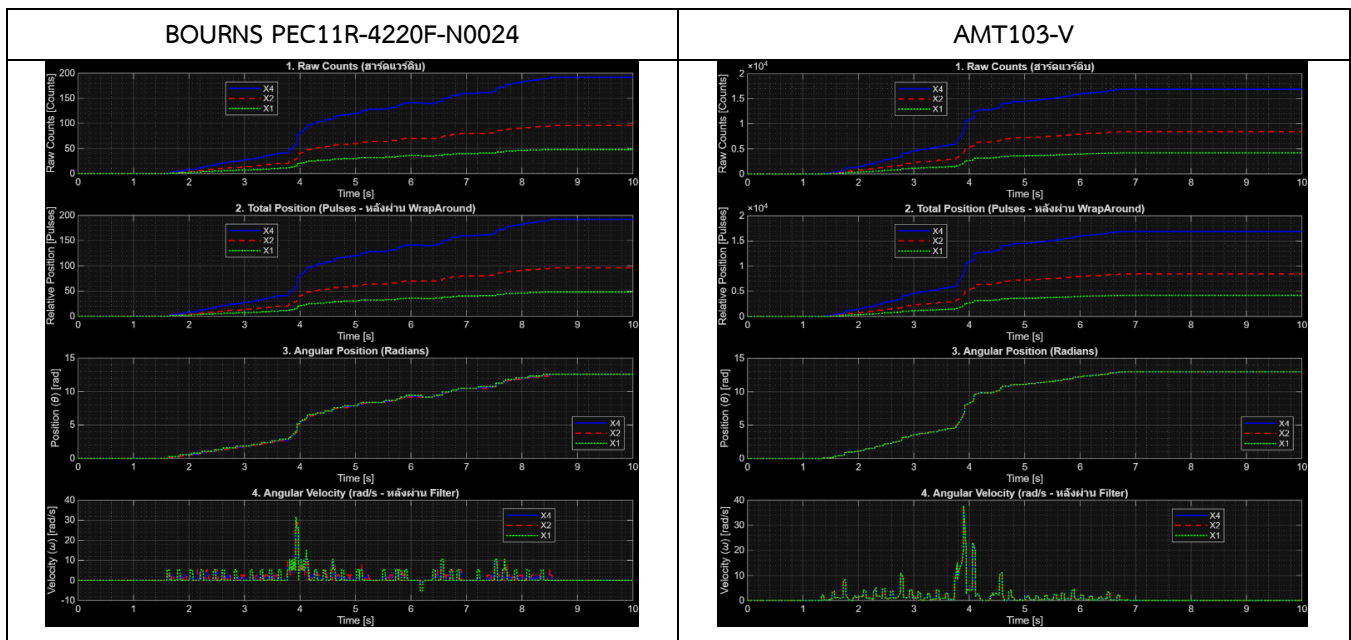
ตารางที่ 12 การตรวจสอบการทำงานของระบบจัดการ Wraparound เมื่อหมุน Encoder ในทิศทางย้อนกลับ



ตารางที่ 13 วิเคราะห์ความสัมพันธ์ระหว่างการเปลี่ยนแปลงตำแหน่งเชิงมุมและความเร็วเชิงมุมเมื่อกลับทิศทางการหมุน



ตารางที่ 14 วิเคราะห์ประสิทธิภาพของวงจรกรองสัญญาณ (Filter) ในการคำนวณความเร็วจากการกระชากของตำแหน่ง



สรุปผลการทดลอง

ความแตกต่างของโมเมนต์ X1, X2 และ X4: จากการเก็บผลการทดลองพบว่า สัญญาณโมเมนต์ X4 ให้จำนวน pulses สะสม สูงที่สุด รองลงมาคือโมเมนต์ X2 และ X1 ตามลำดับ ซึ่งแตกต่างกันแบบเชิงเส้นตามตัวคูณ แต่เมื่อนำค่าพัลส์เหล่านี้ไปคูณกับ Gain เพื่อแปลงเป็น Angular Position กราฟของทั้ง 3 โมเมนต์จะวิ่งทับซ้อนกันที่ตำแหน่งองศาเดียวกัน

การทดสอบการจัดการ Wraparound: เมื่อหมุนแกน Encoder ย้อนกลับผ่านจุดเริ่มต้น ค่า Raw Counts จากฮาร์ดแวร์ เกิดการกระโดดข้ามจาก 0 ไปยังค่า 65,535 ในทันที แต่เมื่อสัญญาณผ่านฟังก์ชันที่เขียนดักไว้ ค่า Relative Position สามารถแสดงผลการนับถอยหลังเป็นค่าติดลบได้อย่างต่อเนื่องโดยไม่มีการกระโดด

การทดสอบทิศทางและความสัมพันธ์ของความเร็ว: เมื่อหมุนแกนไปในทิศทางหนึ่ง กราฟตำแหน่งเชิงมุมมีความชันเป็นบวก และกราฟความเร็วเชิงมุมแสดงค่าอยู่ในช่วงบวก ในช่วงที่หยุดพักมือนิ่ง กราฟตำแหน่งคงที่เป็นเส้นตรงแนวนอน และกราฟความเร็วตกลงมาอยู่ที่ระดับ 0 rad/s พอดี และเมื่อหมุนแกนย้อนกลับ กราฟตำแหน่งเปลี่ยนเป็นความชันติดลบ กราฟความเร็วก็พลิกกลับมาแสดงค่าในช่วงลบเช่นกัน

การทดสอบการกรองสัญญาณ: ในช่วงที่ทำการบิดกระชากแกนอย่างรวดเร็ว กราฟตำแหน่งเกิดความชันสูงชันในระยะเวลาสั้นๆ ส่งผลให้กราฟความเร็วพุ่งทะยานขึ้นเป็นยอดแหลมทันที ส่วนในช่วงการหมุนแกนอย่างช้าๆ กราฟความเร็วมีการเกาะกลุ่มอยู่ในระดับต่ำ แต่พบการแกว่งสวิงขึ้นลง สลับกับศูนย์อย่างต่อเนื่อง ซึ่งอาการแกว่งนี้เห็นได้ชัดเจนแตกต่างกันระหว่าง Encoder ทั้ง 2 รุ่น

อภิปรายผล

การประเมินความละเอียดและสมการแปลงหน่วย: การที่โหมด X4 ให้จำนวนพัลส์สูงสุด ยืนยันสมมติฐานว่าโหมด X4 มีความละเอียดของสัญญาณมากที่สุด เนื่องจากการนับพัลส์ทั้งขอบขาขึ้นและขาลงของทั้งแชนเนล A และ B การที่กราฟ Angular Position ของทั้ง 3 โหมดทับซ้อนกัน เป็นการพิสูจน์ว่าสมการ *Angular Resolution* = $\frac{360^\circ}{PPR \times Multiplier}$ ที่ตั้งเป็น Gain นั้นถูกต้องและสามารถแปลงหน่วยจากความละเอียดที่ต่างกันให้กลับมาเป็นระยะทางกายภาพจริงได้อย่างแม่นยำ

การขดเขยข้อจำกัดทางฮาร์ดแวร์ (Wraparound): ปรากฏการณ์ที่ค่าดิบกระโดดไปที่ 65,535 เกิดจากการเกิด Underflow ซึ่งเป็นข้อจำกัดของ 16-bit Timer อัลกอริทึมจัดการ Overflow/Underflow ใน Simulink ทำงานได้สมบูรณ์ โดยสามารถตรวจจับช่วงรอยต่อและบวกลบค่าขดเขย ทำให้ระบบสามารถวัดและติดตาม Relative position ทางกลศาสตร์ได้อย่างถูกต้อง

ความสัมพันธ์เชิงกลศาสตร์: พฤติกรรมของกราฟสอดคล้องกับสมมติฐานและกฎทางแคลคูลัสอย่างสมบูรณ์ ความเร็วเชิงมุมคืออัตราการเปลี่ยนแปลงของมุมเทียบกับเวลา ($\omega = \frac{d\theta}{dt}$) ดังนั้นเครื่องหมายบวกหรือลบของความเร็วจึงต้องแปรผันตรงกับทิศทางความชันของกราฟตำแหน่งเสมอ การที่ความเร็วเป็นศูนย์เมื่อตำแหน่งไม่เปลี่ยนแปลง ยืนยันว่าระบบแยกแยะทิศทาง Quadrature และสมการหาอนุพันธ์ในโมเดลตอบสนองได้อย่างถูกต้อง

อัตราการตอบสนองและประสิทธิภาพของ Filter: ระบบหาอนุพันธ์มีอัตราการตอบสนองต่อการกระชากของสัญญาณ ที่ไวต่อการเปลี่ยนแปลง สามารถตรวจจับความเร่งจากการกระชากแกนได้โดยไม่มีควมหน่วงที่ผิดปกติ

ข้อเสนอแนะ

- การทดสอบปรากฏการณ์ Overflow ในทิศทางบวก ทำได้ยากเมื่อทดสอบด้วยมือ เนื่องจากต้องสร้างพัลส์สะสมให้ถึงขีดจำกัดสูงสุดของ 16-bit Timer (65,535 pulses) ซึ่งต้องใช้จำนวนรอบการหมุนที่มากเกินไปจนกว่าจะทำได้
เสนอแนะให้ใช้มอเตอร์ไฟฟ้าขับเคลื่อน Encoder แทนการใช้มือหมุน เพื่อให้สามารถปรับรอบความเร็วสูงและทดสอบขอบเขตการ Overflow ได้อย่างรวดเร็วและเห็นภาพชัดเจนยิ่งขึ้น
- การใช้มือหมุนแกน Encoder ทำให้ความเร็วรอบ ไม่สม่ำเสมอ ส่งผลให้กราฟความเร็วเกิดสัญญาณรบกวนและการแกว่งตัว เสนอแนะให้ติดตั้ง Encoder เข้ากับชุดทดสอบแบบมอเตอร์ที่สามารถตั้งค่าความเร็วคงที่ได้ ซึ่งจะช่วยให้การวิเคราะห์ประสิทธิภาพของตัวกรองสัญญาณแบบ Moving Average ในการลดสัญญาณรบกวนมีความแม่นยำและน่าเชื่อถือมากกว่าการประเมินจากการกระชากด้วยมือ

อ้างอิง

- <https://www.haydonkerkpittman.com/learningzone/whitepapers/incremental-encoder-signals>

1.4 Lab Load Cell

การทดลองที่ 1 Single Point Load Cell กับวงจรขยายสัญญาณ INA125 Instrumentation Amplifier

จุดประสงค์

1. เพื่อศึกษาหลักการทำงานของ Strain Gauge, วงจร Wheatstone Bridge และการขยายสัญญาณด้วย INA125
2. เพื่อศึกษาคำนวณค่า Gain และค่าความต้านทาน ที่เหมาะสมเพื่อป้องกันการเกิด Saturation
3. เพื่อเรียนรู้กระบวนการ Signal Conditioning และการเทียบมาตรฐาน (Calibration) สำหรับแปลงค่าอนาล็อกเป็นหน่วยน้ำหนักจริง

สมมติฐาน

1. เมื่อแรงกดบน Load Cell เพิ่มขึ้น ค่าความต้านทานในวงจร Wheatstone Bridge จะเปลี่ยนแปลง ส่งผลให้แรงดันเอาต์พุตแปรผันเป็นเส้นตรงตามน้ำหนัก
2. การตั้งค่า Gain ที่สูงเกินไปจะทำให้สัญญาณ Output เกิดการอิ่มตัว (Saturation) ที่ขอบแรงดันสูงสุด

ตัวแปร

1. ตัวแปรต้น:
 - น้ำหนักหรือมวลที่กดลงบน Load Cell (kg)
2. ตัวแปรตาม:
 - ค่า ADC (Analog-to-Digital Converter) ที่อ่านได้ ตั้งแต่ 0-4095
 - แรงดันไฟฟ้า Output
 - ค่าน้ำหนักที่คำนวณได้ (kg)
3. ตัวแปรควบคุม:
 - แรงดันไฟฟ้า Input
 - บอร์ดประมวลผล Nucleo STM32G474RE

เอกสารและงานวิจัยที่เกี่ยวข้อง

หลักการทำงานของ Load Cell และวงจร Wheatstone Bridge Load Cell ชนิด Single Point เป็นอุปกรณ์ตรวจวัดแรงหรือน้ำหนักโดยอาศัยการเปลี่ยนแปลงทางกลของ Strain Gauge ที่ยึดติดอยู่บนโครงสร้างอลูมิเนียม เมื่อมีแรงมากระทำ โครงสร้างจะเกิดการยืดและหดตัว ส่งผลให้ค่าความต้านทานไฟฟ้าภายใน Strain Gauge เปลี่ยนแปลงไป ในการทดลองนี้ Strain Gauge จำนวน 4 ตัวถูกนำมาต่อกันในรูปแบบ วงจร Wheatstone Bridge (Full-Bridge

Configuration) เพื่อแปลงการเปลี่ยนแปลงความต้านทานเพียงเล็กน้อยให้เป็นความต่างศักย์ไฟฟ้าอนาล็อก ($V_{IN}^+ - V_{IN}^-$) ซึ่งแปรผันเป็นเส้นตรงตามแรงหรือน้ำหนักที่กดลงบน Load Cell

คุณลักษณะของวงจรขยายไอซี INA125 เนื่องจากสัญญาณแรงดันไฟฟ้าเอาต์พุตที่ออกจากวงจร Wheatstone Bridge มีขนาดเล็กมาก (ระดับมิลลิโวลต์) จึงจำเป็นต้องผ่านกระบวนการปรับแต่งสัญญาณ (Signal Conditioning) ด้วย ไอซีขยายสัญญาณความแม่นยำสูง INA125 ซึ่งเป็น Instrumentation Amplifier แบบ 2 Op-amps Differential Amplifier ที่มีจุดเด่นคือมีแหล่งจ่ายแรงดันอ้างอิงเชิงเส้น (Precision Voltage Reference) ในตัว สำหรับใช้เป็นแรงดันป้อนเข้า (Excitation Voltage) ให้กับวงจร Bridge

โครงสร้างการทำงานของ 2 Op-amps Differential Amplifier ภายใน INA125

โครงสร้างภายในของ INA125 ใช้การต่อวงจรรวมแบบ Differential Amplifier ชนิด 2 Op-amps (A1 และ A2) ร่วมกับเครือข่ายความต้านทานที่มีความเที่ยงตรงสูง เพื่อขยายสัญญาณส่วนต่างแรงดันขนาดเล็ก $V_{IN}^+ - V_{IN}^-$ ที่ออกจากวงจร Wheatstone Bridge ดังนี้

1. Op-amp ตัวแรก (A_1): ทำหน้าที่เป็นตัวรับสัญญาณแรงดันอินพุตแบบไม่กลับเฟส (Non-inverting Input) ร่วมกับขารับความต้านทานขยายภายนอก R_G ซึ่งจะทำให้การป้อนกลับสัญญาณเพื่อเริ่มขยายแรงดันส่วนต่างในขั้นแรก (First Gain Stage)
2. Op-amp ตัวที่สอง (A_2): ทำหน้าที่ขยายสัญญาณส่วนต่างในขั้นถัดมา พร้อมทั้งทำหน้าที่ปราบปรามและหักล้างสัญญาณรบกวนชนิดโหมดร่วม (Common-mode Rejection Ratio: CMRR) ที่เกิดขึ้นเท่าๆ กันบนสายสัญญาณทั้งสองเส้น ออกไปจากระบบ
3. การแปลง Output: วงจรจะรวมสัญญาณขยายออกมาเป็นแรงดันไฟฟ้าแบบอ้างอิงเทียบกราวด์ (Single-ended Output Voltage) ส่งต่อไปยังขั้ว ADC ของไมโครคอนโทรลเลอร์ได้อย่างเสถียร โดยมีอัตราการขยายสัญญาณ

เมื่อสัญญาณส่วนต่างแรงดันถูกขยาย แรงดันไฟฟ้าเอาต์พุต (V_{out}) จะคำนวณได้จากสมการ

$$V_{out} = (V_{IN}^+ - V_{IN}^-) \times G$$

ขนาดการขยายสัญญาณ (Gain: G) ของ INA125 สามารถกำหนดและปรับแต่งได้ผ่านการต่อค่าความต้านทานภายนอก R_G เพียงตัวเดียว ระหว่างพิน R_G ทั้งสองพิน โดยมีความสัมพันธ์ตามสมการ

$$G = 4 + \frac{60 \text{ k}\Omega}{R_G}$$

การเลือกค่า R_G ที่เหมาะสมจะช่วยขยายสัญญาณให้อยู่ในช่วงที่ตัวแปลงสัญญาณอนาล็อกเป็นดิจิทัล (ADC) ของบอร์ดไมโครคอนโทรลเลอร์สามารถอ่านได้อย่างมีประสิทธิภาพสูงสุด และป้องกันไม่ให้เกิดสถานะสัญญาณอิ่มตัว (Saturation) ที่ขอบแรงดันไฟเลี้ยง (V_{supply})

สาเหตุความคลาดเคลื่อนและการสูญเสียสัญญาณในระบบวัด ในการตรวจวัดด้วยวงจร Bridge สัญญาณเอาต์พุตอาจเกิดความคลาดเคลื่อน (%Error) จากหลายปัจจัย ได้แก่ ความต้านทานของสายนำสัญญาณ (Leadwire Resistance) ซึ่งทำให้เกิดแรงดันตกคร่อมในสาย ส่งผลให้แรงดันป้อนเข้าวงจร Bridge ลดลง หรือเกิดความไม่สมดุลของวงจร รวมถึงปัจจัยด้านความเค้นตกค้างในโครงสร้างอลูมิเนียม (Mechanical Creep), สัญญาณรบกวนทางไฟฟ้า (Electrical Noise) และการเปลี่ยนแปลงของอุณหภูมิรอบข้าง จึงต้องมีการปรับชดเชยและเทียบมาตรฐาน (Calibration) ในโปรแกรมก่อนนำค่าไปใช้งานจริง

ขั้นตอนการดำเนินงาน

1. การติดตั้งและทดสอบสถานะอิ่มตัว (Saturation Test)
 - ติดตั้งบอร์ด Nucleo STM32-G474RE เข้ากับวงจรขยาย INA125 และ Single Point Load Cell (YZC-131A)
 - เชื่อมต่อสาย VCC, GND และ OUT เข้ากับพอร์ต ADC ที่กำหนด รวมถึงต่อค่าความต้านทาน R_G เริ่มต้น
 - ทดลองใส่น้ำหนักเพิ่มขึ้นเรื่อย ๆ จนกระทั่งค่าแรงดันไฟฟ้า Output (V_{out}) หรือค่า ADC หยุดการเปลี่ยนแปลงที่ระดับแรงดันสูงสุด (ประมาณ 3.3V) เพื่อยืนยันสถานะการเกิด Saturation ของระบบ
2. การปรับตั้งค่าความต้านทาน เพื่อป้องกัน Saturation
 - คำนวณหาค่า Gain (G) และปรับเปลี่ยนค่าความต้านทาน R_G (ผ่าน Trimpot 100 k Ω) ให้มีค่าสูงขึ้นเพื่อลดระดับ Gain ลง
 - ตรวจสอบให้แน่ใจว่าเมื่อใส่น้ำหนักพิกัดสูงสุด สัญญาณ Output จะไม่พุ่งสูงจนชนขอบแรงดันไฟเลี้ยง เพื่อป้องกันไม่ให้เกิด Saturation ตลอดช่วงการทดลอง
3. ขั้นตอนการเก็บข้อมูลการใส่น้ำหนัก
 - เริ่มใส่น้ำหนักเพิ่มขึ้นทีละ 1kg บน Load Cell ตามลำดับ
 - ณ ทุก ๆ ระดับน้ำหนัก ให้รันโปรแกรมผ่าน MATLAB Script โดยสั่งงาน Simulink Model เป็นเวลา 10 วินาที เพื่ออ่านค่าดิจิทัล ADC (0–4095) และใช้บล็อก Gain แปลงค่าเป็นแรงดันไฟฟ้าเอาต์พุต V_{OUT} (V) ร่วมกับการทำ Sample Averaging เพื่อลดสัญญาณรบกวน (Noise)
 - ทำการทดลองใส่น้ำหนักตามขั้นตอนเดิมซ้ำทั้งหมดจำนวน 3 รอบ เพื่อนำค่าแรงดันไฟฟ้าเอาต์พุตที่วัดได้ในแต่ละรอบมาคำนวณหาค่าเฉลี่ยรวม (Voltage Average)
 - นำข้อมูลน้ำหนักอ้างอิง kg บนแกน X และค่าแรงดันไฟฟ้าเอาต์พุตเฉลี่ย V_{OUT} บนแกน Y ไปพล็อตสร้างกราฟความสัมพันธ์ลงในโปรแกรม MATLAB
 - คำนวณหาค่าความชัน (Slope: m) และค่าจุดตัดแกน Y (Intercept: V_Q) ผ่านสมการการถดถอยเชิงเส้น (Linear Regression) คำนวณได้จากสมการ

$$V_{out} = m \cdot Weight + V_Q$$

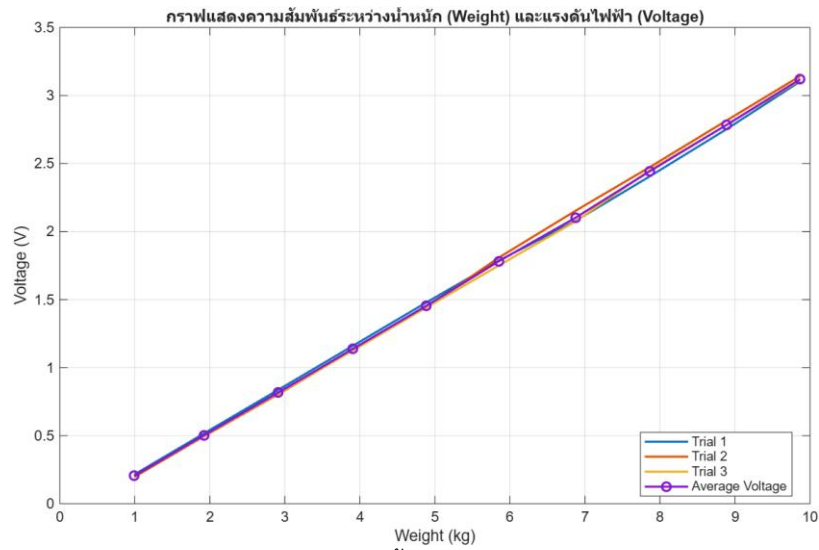
4. การเปรียบเทียบและยืนยันความถูกต้องของสมการ

- นำค่าความชัน (m) และ V_Q ที่ได้จากการคำนวณ ย้อนกลับมาใส่ในสมการเพื่อคำนวณหาค่าน้ำหนักย้อนกลับ คำนวณได้จากสมการ

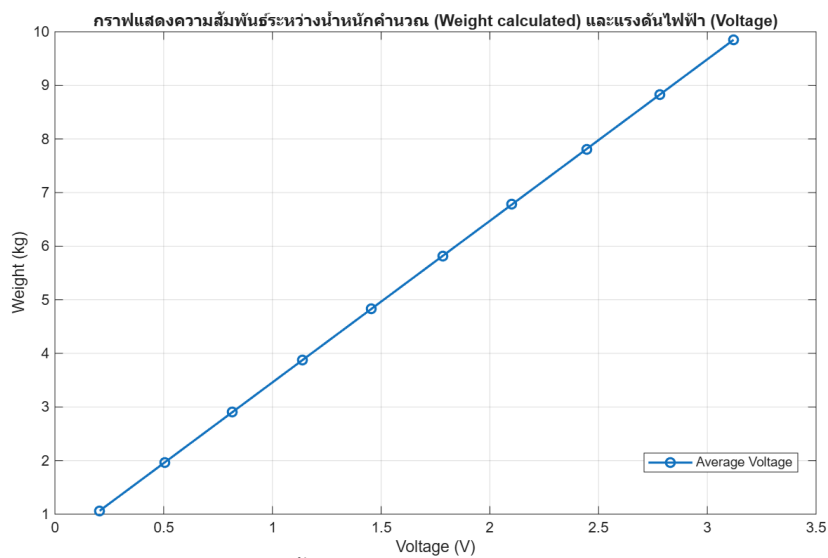
$$Weight_{calculated} = \frac{V_{out} - V_Q}{m}$$

- นำค่าน้ำหนักที่คำนวณได้จากสมการไปเปรียบเทียบกับค่าน้ำหนักจริงที่ใส่ลงไป เพื่อตรวจสอบดูว่าค่าแรงดันไฟฟ้าเอาต์พุต (V_{out}) ที่เกิดขึ้น ตอบสนองตรงตามค่าน้ำหนักจริงหรือไม่ พร้อมทั้งวิเคราะห์ค่าเปอร์เซ็นต์ความคลาดเคลื่อน (%Error) และความเป็นเชิงเส้น (Linearity)

ผลการทดลอง



รูปที่ 3 กราฟแสดงความสัมพันธ์ระหว่างน้ำหนัก (Weight) และแรงดันไฟฟ้า (Voltage)



รูปที่ 4 กราฟแสดงความสัมพันธ์ระหว่างน้ำหนักคำนวณ (Weight calculated) และแรงดันไฟฟ้า (Voltage)

สรุปผลการทดลอง

การคำนวณอัตราขยายสัญญาณและการป้องกันภาวะ Saturation จากการต่อค่าความต้านทานภายนอก $R_G = 70.2 \, \Omega$ เข้ากับพิน R_G ของไอซีขยายสัญญาณ INA125 สามารถคำนวณหาอัตราขยายสัญญาณ (Gain: G) ได้จากสมการ

$$G = 4 + \frac{60 \, k\Omega}{R_G}$$

$$G = 858.70$$

เมื่อพิจารณาสัญญาณอินพุตส่วนต่างแรงดันสูงสุดจาก Load Cell ($\Delta V = V_{IN}^+ - V_{IN}^- = 3.2mV$) สัญญาณแรงดันเอาต์พุตเฉพาะส่วนการขยาย (ΔV_{out}) จะมีค่าเท่ากับ

$$V_{out} = (V_{IN}^+ - V_{IN}^-) \times G$$

$$V_{out} = 2.74784V$$

การคำนวณหาค่า Offset แรงดันเริ่มต้น (V_Q) จากการทดลอง จากรูปที่ 5 กราฟแสดงความสัมพันธ์ระหว่างน้ำหนัก (Weight) และแรงดันไฟฟ้า (Voltage)

เนื่องจากขั้นตอนการทดลองเริ่มใส่น้ำหนักทดสอบตั้งแต่ 0.993kg ถึง 9.870kg จึงทำการวิเคราะห์หาค่า Offset แรงดันอ้างอิง ณ สภาวะไม่ใส่น้ำหนัก (0kg) หรือค่า V_Q ผ่านวิธีการถดถอยเชิงเส้น (Linear Regression) จากชุดข้อมูลการทดลองทั้งหมด 10 จุด จากสมการความสัมพันธ์เชิงเส้น

$$V_{out} = m \cdot Weight + V_Q$$

เมื่อแทนค่าความชันของระบบ $m = 0.33177V/kg$ และค่าน้ำหนักพร้อมแรงดันเอาต์พุต ณ จุดแรก (Weight = 0.993kg, $V_{out} = 0.196147V$)

$$V_Q = -147.25 mV$$

พบว่าได้ค่าแรงดัน V_Q ทางทฤษฎีเท่ากับ -0.14725V (-147.25mV) ซึ่งใช้เป็นค่าชดเชยสมดุลในสมการปรับเทียบมาตรฐาน (Calibration) ต่อไป

ความสัมพันธ์เชิงเส้นและสมการ Calibration

แรงดันเอาต์พุต (V_{out}) มีความสัมพันธ์เชิงเส้นแปรผันตรงกับน้ำหนัก จากการวิเคราะห์ชุดข้อมูล 10 จุด (0.993kg ถึง 9.870kg) ได้ค่าความชัน $m = 0.33177V/kg$ และ $V_Q = -0.14725V$ นำมาจัดรูปเป็น สมการ Calibration

$$Weight_{calculated} = \frac{V_{out} + 0.14725}{0.33177}$$

ความคลาดเคลื่อนของระบบ เมื่อนำสมการปรับเทียบ มาคำนวณย้อนกลับหาค่าน้ำหนัก พบว่าระบบมี % Relative Error เฉลี่ยเพียง 0.92% และมี % Full Scale Error สูงสุดไม่เกิน 0.56% แสดงว่าระบบมีความแม่นยำและความน่าเชื่อถือสูงตลอดช่วงการวัด โดยความคลาดเคลื่อนสัมพัทธ์ส่วนใหญ่จะเกิดขึ้นในจุดน้ำหนักน้อย (0.993kg) และลดลงอย่างมีนัยสำคัญเมื่อน้ำหนักเพิ่มขึ้น

อภิปรายผล

การขยายสัญญาณและการป้องกันสภาวะ Saturation

- วงจร Wheatstone Bridge บน Load Cell สร้างสัญญาณความต่างศักย์ระดับมิลลิโวลต์ ($\Delta V \approx 3.2 \text{ mV}$) เมื่อผ่านวงจรขยาย INA125 โดยใช้ความต้านทาน $R_G = 70.2 \Omega$ ทำให้ได้อัตราขยายสัญญาณ (Gain: G) สูงถึง 858.70 เท่า
- การเลือกค่า $R_G = 70.2 \Omega$ ระดับนี้มีความเหมาะสมอย่างยิ่ง เนื่องจากสามารถขยายสัญญาณแรงดันเพียงเบนสูงสุด (ΔV_{out}) ออกมาได้ประมาณ 2.75V ซึ่งอยู่ภายในช่วงแรงดันทำงาน 0–3.3V ของ ADC บอร์ด Nucleo STM32 ทำให้ไม่เกิดสภาวะสัญญาณอิ่มตัว (Saturation) ที่ขอบแรงดันไฟเลี้ยง และใช้ประโยชน์จากความละเอียดของ ADC ได้อย่างเต็มประสิทธิภาพ
- ทั้งนี้ จากการทดสอบสภาวะอิ่มตัว (Saturation Test) โดยการใส่น้ำหนักกดเกินพิกัดย่านเชิงเส้น ($> 10\text{kg}$) แม้วงจรขยาย INA125 จะรับไฟเลี้ยง $V_{CC} = 5V$ แต่เมื่อแรงดันเอาต์พุตขยายเกิน 3.3V สัญญาณที่อ่านได้ผ่านพอร์ต ADC ของไมโครคอนโทรลเลอร์จะพุ่งขึ้นไปอิ่มตัวและถูกจำกัด (Clipped) อยู่ที่ระดับแรงดันอ้างอิงสูงสุด 3.3V (หรือค่านับดิจิทัลอิ่มตัวที่ $ADC = 4095$) ยืนยันถึงขอบเขตย่านการวัดสูงสุด (Full Scale Range) ของระบบการอ่านค่า

การคำนวณค่า Offset (V_Q) และสมการปรับเทียบมาตรฐาน (Calibration)

- จากการวิเคราะห์ความสัมพันธ์เชิงเส้นด้วยวิธี Linear Regression บนชุดข้อมูล 10 จุด (0.993 kg ถึง 9.870 kg) ได้สมการการถดถอย คือ $V_{out} = 0.33177 \cdot Weight_{calculated} - 0.14725$
- ค่าจุดตัดแกน Y ($V_Q = -0.14725V$) แสดงถึงแรงดัน Offset ทางทฤษฎี ณ สภาวะไม่ใส่น้ำหนัก ซึ่งเกิดจากความไม่สมดุลเริ่มต้นของวงจร Bridge และน้ำหนักของโครงสร้าง Load Cell เอง การนำค่า V_Q นี้ไปจัดรูปเป็นสมการ Calibration $Weight_{calculated} = \frac{V_{out} + 0.14725}{0.33177}$ ช่วยให้สามารถแปลงค่าแรงดันไฟฟ้าที่อ่านได้กลับมาเป็นค่าน้ำหนักจริงได้อย่างแม่นยำ

ความแม่นยำและความคลาดเคลื่อนของระบบ

- ผลการคำนวณย้อนกลับพบว่าระบบมี % Relative Error เฉลี่ยต่ำเพียง 0.92% และมี % Full Scale Error สูงสุดไม่เกิน 0.56%

ข้อเสนอแนะ

- การชดเชยผลกระทบจากอุณหภูมิและความเค้นตกค้าง โดย Strain Gauge และโครงสร้าง อลูมิเนียมไวต่อการเปลี่ยนแปลงอุณหภูมิรอบข้าง ในการใช้งานจริงระยะยาว ควรมีการติดตั้งเซ็นเซอร์อุณหภูมิเพื่อทำ Temperature Compensation ในโค้ด และคำนึงถึงปรากฏการณ์ Mechanical Creep เมื่อต้องรับน้ำหนักแช่ไว้นาน ๆ

- สัญญาณจาก Strain Gauge และวงจรขยาย Gain สูง ไวต่อสัญญาณรบกวนทางไฟฟ้า (Electrical Noise) จากสายไฟและแหล่งจ่าย ควรเพิ่ม C-Filter ในขนานกับขา R_G หรือออกแบบวงจร Active Low-Pass Filter คัดแยกความถี่รบกวนออกก่อนส่งเข้าขั้ว ADC เพื่อเพิ่มความเสถียรของตัวเลขน้ำหนักในทศนิยมตำแหน่งที่สูงขึ้น

อ้างอิง

- https://www.ti.com/lit/ds/symlink/ina125.pdf?ts=1791046032045&ref_url=https%253A%252F%252Fwww.google.com%252F
- https://www.phidgets.com/docs/Load_Cell_Guide
- <https://micro-measurements.com/knowledge-base/technical-notes>

ภาคผนวก

1. Lab 1.1 Potentiometer

1.1 โปรแกรมรัน Simulink 10 วินาที และบันทึกข้อมูลดิบพร้อมค่าเฉลี่ยลงไฟล์ .csv

ในการทดลองใช้ MATLAB Script ในการควบคุมการรัน Simulink Model (Simulink_Lab1.1) เป็นเวลา 10 วินาที ณ แต่ละจุดวัด โดยสคริปต์จะทำการดึงข้อมูล Time-series Raw Data และคำนวณค่าแรงดันไฟฟ้าเฉลี่ย (Sample Averaging) เพื่อลดสัญญาณรบกวน (Noise Reduction) พร้อมบันทึกข้อมูลลงไฟล์ .csv 2 รูปแบบ คือ ไฟล์ตารางค่าเฉลี่ยสะสม (Potentiometer_Average_[Linear/Rotary].csv) และ ไฟล์ตารางข้อมูลดิบแยกรายรอบ (Potentiometer_RawData_[Linear/Rotary]_Run%d.csv)

1.1.1 สคริปต์สำหรับ Linear Potentiometer (Type A และ Type B)

```
1 % 1. โหลดโมเดล
2 model_name = 'Simulink_Lab1.1';
3 load_system(model_name);
4
5 % 2. ตรวจสอบและล้างตัวแปร
6 if ~exist('History_Avg_Run', 'var')
7     % ตัวแปรเก็บค่าเฉลี่ย
8     History_Avg_Run = [];
9     History_Avg_A1 = [];
10    History_Avg_B1 = [];
11    Run_Count = 0;
12 end
13
14 disp('--- โปรแกรมเก็บข้อมูล A1 และ B1 (ส่งออก CSV: ค่าเฉลี่ย 1 ไฟล์ + Raw Data แยกตามรอบ) ---');
15
16 % 3. เริ่มวนลูป
17 while true
18     str = input('\nกด Enter รัน 10 s (พิมพ์ c เพื่อล้างค่า, พิมพ์ q เพื่อออก): ', 's');
19
20     if strcmpi(str, 'q')
21         clear History_Avg_Run History_Avg_A1 History_Avg_B1 Run_Count;
22         disp('❏ ออกจากโปรแกรม และล้างค่าเรียบร้อยแล้ว!');
23         break;
24     elseif strcmpi(str, 'c')
25         History_Avg_Run = []; History_Avg_A1 = []; History_Avg_B1 = [];
26         Run_Count = 0;
27         disp('❏ ล้างข้อมูลเก่าออกหมดแล้ว! พร้อมรันรอบ 1 ใหม่!');
28         continue;
29     end
30
31     Run_Count = Run_Count + 1;
32     fprintf('\n❏ กำลังรัน Simulink รอบที่ %d...\n', Run_Count);
33
34
35     % รัน 10 วินาที
36     out = sim(model_name, 'StopTime', '10');
37
38     % ---- ดึงค่า Raw Data ----
39     % ดึงค่าจาก out_A1 และ out_B1 ใน Simulink
40     time_sec = out.out_A1.Time;
41     A1_data = out.out_A1.Data;
42     B1_data = out.out_B1.Data;
43
44     % ---- ดึงค่า Average Data ----
45     cur_mean_A1 = mean(A1_data);
46     cur_mean_B1 = mean(B1_data);
47
48     % ส่งค่าเก็บค่าเฉลี่ย
49     History_Avg_Run(end+1, 1) = Run_Count;
50     History_Avg_A1(end+1, 1) = cur_mean_A1;
51     History_Avg_B1(end+1, 1) = cur_mean_B1;
52
53     fprintf('❏ ค่าเฉลี่ยรอบที่ %d -> A1: %.4f V | B1: %.4f V\n', ...
54         Run_Count, cur_mean_A1, cur_mean_B1);
55
56     % 4. บันทึกเป็นไฟล์ CSV (แยกตามรอบ)
57
58     % 4.1 ไฟล์ที่ 1: ตารางค่าเฉลี่ย (ส่งค่าไฟล์แล้ว)
59     T_avg = table(History_Avg_Run, History_Avg_A1, History_Avg_B1, ...
60         'VariableNames', {'Run_Number', 'Avg_Voltage_A1', 'Avg_Voltage_B1'});
61     writetable(T_avg, 'Potentiometer_Average_Linear.csv');
62
63     % 4.2 ไฟล์ที่ 2, 3, 4... : ตารางข้อมูลดิบ (ส่งค่าไฟล์ทีละรอบ)
64     T_raw_current = table(time_sec, A1_data, B1_data, ...
65         'VariableNames', {'Time_sec', 'Voltage_A1', 'Voltage_B1'});
66
67     % ส่งค่าไฟล์ดิบกับนามสกุลตามรอบ เช่น Potentiometer_RawData_Run1.csv
68     filename_raw = sprintf('Potentiometer_RawData_Linear_Run%d.csv', Run_Count);
69     writetable(T_raw_current, filename_raw);
70
71     fprintf('❏ บันทึกไฟล์เสร็จ! (ไฟล์ชื่อ %s)\n', filename_raw);
72
73 end
```

สคริปต์นี้ถูกออกแบบมาเพื่อเก็บสัญญาณอนาล็อกขา A3 [A1](Linear Type A) และ A4 [B1](Linear Type B) โดยมีขั้นตอนการทำงานดังนี้

1. การโหลดแบบจำลองและการตั้งค่าเริ่มต้น (บรรทัดที่ 1-12): สคริปต์ทำการโหลดระบบ Simulink_Lab1.1 เข้าสู่ความจำ MATLAB และตรวจสอบตัวแปรสำหรับสะสมค่าเฉลี่ย ได้แก่ History_Avg_Run, History_Avg_A1 และ History_Avg_B1
2. การควบคุมลูปการทดลอง (บรรทัดที่ 17-31): ใช้ลูป while true เพื่อรอรับคำสั่งจากผู้ใช้งาน โดยเมื่อกดปุ่ม Enter สคริปต์จะเริ่มนับรอบการทดลอง (Run_Count), ปุ่ม c ใช้สำหรับล้างค่าข้อมูลดิบเพื่อเริ่มนับรอบใหม่ และปุ่ม q ใช้สำหรับคืนค่าตัวแปรพร้อมออกจากโปรแกรม
3. การสั่งรันและสุ่มเก็บข้อมูล 10 วินาที (บรรทัดที่ 33-42): สั่งรัน Simulink Model เป็นเวลา 10 วินาที เพื่อดึงข้อมูลสัญญาณ Time-series ของเวลา (time_sec) และค่าแรงดันไฟฟ้าดิบ (A1_data และ B1_data) ณ ตำแหน่งที่เลื่อนทดสอบ

4. การประมวลผลค่านวนค่าเฉลี่ย (บรรทัดที่ 44-54): นำข้อมูลดิบมาหาค่าเฉลี่ยด้วยฟังก์ชัน mean() เพื่อลดความคลาดเคลื่อนจากสัญญาณรบกวน (Noise Reduction) แล้วสะสมลงในตัวแปรประวัติการรัน

5. การส่งออกข้อมูลเป็นไฟล์ CSV (บรรทัดที่ 58-73):

- บันทึกค่าเฉลี่ยรวมสะสมลงในไฟล์ Potentiometer_Average_Linear.csv ผ่านตาราง T_avg
- บันทึกข้อมูลดิบรายละเอียดของการรันรอบนั้น ๆ ลงในไฟล์ Potentiometer_RawData_Linear_Run%d.csv ผ่านตาราง T_raw_current เพื่อนำไปวิเคราะห์ต่อ

1.1.2 สคริปต์สำหรับ Rotary Potentiometer (Type A, Type B และ Type C)

```
1 % 1. โหลดโมเดล
2 model_name = 'Simulink_Lab1.1';
3 load_system(model_name);
4
5 % 2. ตรวจสอบและสร้างตัวแปร
6 if ~exist('History_Avg_Run', 'var')
7     % ตัวแปรเก็บค่าเฉลี่ย
8     History_Avg_Run = [];
9     History_Avg_A = [];
10    History_Avg_B = [];
11    History_Avg_C = [];
12    Run_Count = 0;
13 end
14
15 disp('--- โปรแกรมเก็บข้อมูล (ส่งออก CSV: ค่าเฉลี่ย 1 ไฟล์ + Raw Data แยกไฟล์ตามรอบ) ---');
16
17 % 3. เริ่มวนลูป
18 while true
19     str = input('\nกด Enter รัน 10 ร (พิมพ์ c เพื่อล้างค่า, พิมพ์ q เพื่อออก): ', 's');
20
21     if strcmpi(str, 'q')
22         clear History_Avg_Run History_Avg_A History_Avg_B History_Avg_C Run_Count;
23         disp('ออกจากโปรแกรม และล้างค่าเรียบร้อยแล้ว!');
24         break;
25     elseif strcmpi(str, 'c')
26         History_Avg_Run = []; History_Avg_A = []; History_Avg_B = []; History_Avg_C
27         = [];
28         Run_Count = 0;
29         disp('ล้างข้อมูลเก่าออกหมดแล้ว! พร้อมรันรอบ 1 ใหม่!');
30         continue;
31     end
32
33     Run_Count = Run_Count + 1;
34     fprintf('\nกำลังรัน Simulink รอบที่ %d...\n', Run_Count);
35
36     % ล้าง 10 วินาที
37     out = sim(model_name, 'StopTime', '10');
38
39     % --- จัดการ Raw Data ---
40     time_sec = out.out.A.Time;
41     v1_data = out.out.A.Data;
42     v2_data = out.out.B.Data;
43     v3_data = out.out.C.Data;
44
45     % --- จัดการ Average Data ---
46     cur_mean_A = mean(v1_data);
47     cur_mean_B = mean(v2_data);
48     cur_mean_C = mean(v3_data);
49
50     % สะสมค่าเฉลี่ย
51     History_Avg_Run(end+1, 1) = Run_Count;
52     History_Avg_A(end+1, 1) = cur_mean_A;
53     History_Avg_B(end+1, 1) = cur_mean_B;
54     History_Avg_C(end+1, 1) = cur_mean_C;
55
56     fprintf('ค่าเฉลี่ยรอบที่ %d -> A: %.4f V | B: %.4f V | C: %.4f V\n', ...
57           Run_Count, cur_mean_A, cur_mean_B, cur_mean_C);
58
59     % 4. บันทึกเป็นไฟล์ CSV (แยกตามรอบ)
60     % =====
61
62     % 4.1 ไฟล์ที่ 1: ตารางค่าเฉลี่ย (ตัวแปร: ค่าเฉลี่ย)
63     T_avg = table(History_Avg_Run, History_Avg_A, History_Avg_B, History_Avg_C, ...
64                 'VariableNames', {'Run_Number', 'Avg.Voltage.A', 'Avg.Voltage.B',
65                 'Avg.Voltage.C'});
66     writetable(T_avg, 'Potentiometer_Average_Rotary.csv');
67
68     % 4.2 ไฟล์ที่ 2, 3, 4... : ตารางข้อมูลดิบ (สำหรับไฟล์ข้อมูลดิบ)
69     T_raw_current = table(time_sec, v1_data, v2_data, v3_data, ...
70                         'VariableNames', {'Time_sec', 'Voltage.A', 'Voltage.B', 'Voltage.C'});
71
72     % ชื่อไฟล์ที่จัดเก็บข้อมูลรวมแบบ Potentiometer_RawData_Run1.csv
73     filename_raw = sprintf('Potentiometer_RawData_Rotary_Run%d.csv', Run_Count);
74     writetable(T_raw_current, filename_raw);
75
76     fprintf('บันทึกไฟล์สำเร็จ! (ไฟล์ %s)\n', filename_raw);
77 end
```

สคริปต์ในภาพถูกพัฒนาต่อยอดจากสคริปต์ Linear เพื่อรองรับการเก็บข้อมูลสัญญาณอนาล็อกพร้อมกัน 3 ช่องสัญญาณ จาก Rotary Potentiometer ทั้ง 3 ชนิด ได้แก่ Type A Logarithmic [ขา A0](ช่อง A), Type B Linear [ขา A1](ช่อง B) และ Type C Inverse Logarithmic [ขา A2](ช่อง C) โดยมีขั้นตอนการทำงานดังนี้

1. การจองพื้นที่ตัวแปรสะสม 3 ช่องสัญญาณ (บรรทัดที่ 1-13): ประกาศตัวแปรเก็บค่าเฉลี่ยสะสมแยกตามชนิด ได้แก่ History_Avg_A, History_Avg_B และ History_Avg_C
2. การรับคำสั่งควบคุมลูปรการทำงาน (บรรทัดที่ 18-31): รองรับระบบโต้ตอบกับผู้ใช้งานผ่านคีย์บอร์ด (Enter เพื่อรัน, c เพื่อ Clear ข้อมูลเก่า, q เพื่อ Quit ล้างตัวแปรและปิดลูป) เช่นเดียวกับสคริปต์ Linear

3. การดึงสัญญาณดิบ 3 ช่องทาง (บรรทัดที่ 33-42): สั่งรัน Simulink Model เป็นเวลา 10 วินาที แล้วอ่านค่า Time-series จากบล็อก out_A (v1_data), out_B (v2_data) และ out_C (v3_data)
4. การคำนวณค่าเฉลี่ยรักษาระดับสัญญาณ (บรรทัดที่ 44-55): คำนวณค่าเฉลี่ยแรงดันไฟฟ้าของแต่ละ Potentiometer (cur_mean_A, cur_mean_B, cur_mean_C) เพื่อกรองสัญญาณรบกวน
5. การบันทึกไฟล์ CSV แยกโครงสร้าง (บรรทัดที่ 59-74):
 - อัปเดตตารางค่าเฉลี่ยรวมของทั้ง 3 ชนิดลงในไฟล์ Potentiometer_Average_Rotary.csv
 - สร้างไฟล์บันทึกข้อมูลดิบรายการรอบในรูปแบบ Potentiometer_RawData_Rotary_Run%d.csv เพื่อใช้ในการพล็อตเปรียบเทียบ Taper Curve กับ Datasheet

1.2 สคริปต์สำหรับการโหลดไฟล์ csv และพล็อตแสดงผลกราฟ

หลังจากได้ไฟล์ข้อมูลสะสมจากการทดลอง สคริปต์ในส่วนนี้จะทำหน้าที่อ่านข้อมูลจากไฟล์ .csv นำค่าแรงดันไฟฟ้าและตำแหน่ง/องศาการหมุนมาคำนวณและพล็อตสร้างกราฟความสัมพันธ์เปรียบเทียบระหว่าง Trial และค่าเฉลี่ยรวม

1.2.1 สคริปต์สำหรับ Linear Potentiometer (Type A และ Type B)

```

1 % 1. อ่านข้อมูลจากไฟล์ CSV
2 data = readtable('Data_Lab1.1.csv');
3
4 % 2. กำหนดแกน X และแกน Y จากหัวคอลัมน์ในตาราง
5 % ใช้ Run_Number เป็นแกน X (Rotation)
6 distance = data.Var23;
7
8 % ดึงค่าเฉลี่ยของแต่ละช่องมาเป็นแกน Y
9
10 avg_A1_1 = data.Var24;
11 avg_A1_2 = data.Var25;
12 Avg_A1 = data.Var26;
13
14 avg_B1_1 = data.Var28;
15 avg_B1_2 = data.Var29;
16 Avg_B1 = data.Var30;
17
18 % 3. สร้างหน้าต่างกราฟ
19 figure;
20
21 % 4. พล็อตกราฟเส้น
22
23 plot(distance, avg_A1_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
24 hold on;
25 plot(distance, avg_A1_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
26 plot(distance, Avg_A1, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
27
28 % plot(distance, avg_B1_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
29 % hold on;
30 % plot(distance, avg_B1_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
31 % plot(distance, Avg_B1, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
32
33 % 5. ตกแต่งกราฟให้สวยงามและอ่านง่าย
34 title(sprintf('กราฟแสดงความสัมพันธ์ระหว่างระยะทางการเลื่อน (Distance) และค่าเฉลี่ยแรงดันไฟฟ้า (Voltage Avg.)\nของ Linear Potentiometer Type A'));
35 xlabel('Distance (cm)');
36 ylabel('Voltage Avg. (V)');
37
38 % ใส่คำอธิบายเส้นกราฟ
39 legend('Trial 1', 'Trial 2', 'Average Voltage A', 'Location', 'best');
40 % legend('Trial 1', 'Trial 2', 'Average Voltage B', 'Location', 'best');
41
42 % เปิดตารางกริด
43 grid on;
44 hold off;

```

สคริปต์ในภาพทำหน้าที่อ่านไฟล์ Data_Lab1.1.csv เพื่อนำค่าระยะทาง (distance) และแรงดันไฟฟ้าเฉลี่ยของ Linear Potentiometer Type A และ Type B จากแต่ละรอบการทดลอง (avg_A1_1, avg_A1_2, Avg_A1) มาพล็อตเปรียบเทียบแนวโน้มความเป็นเชิงเส้น (Linearity) พร้อมใส่ชื่อแกน (xlabel/ylabel), ชื่อกราฟ (title) และคำอธิบายเส้นกราฟ (legend) เพื่อเปรียบเทียบค่าระหว่าง Trial 1, Trial 2 และค่าเฉลี่ยรวมอย่างชัดเจน

1.2.2 สคริปต์สำหรับ Rotary Potentiometer (Type A, Type B และ Type C)

```
1 % 1. อ่านข้อมูลจากไฟล์ CSV
2 data = readtable('Data_Lab1.1e.csv');
3
4 % 2. กำหนดแกน X และแกน Y จากหัวคอลัมน์ในตาราง
5 % ใช้ Run_Number เป็นแกน X (Rotation)
6 rotation = data.Var2;
7
8 % ตั้งค่าเฉลี่ยของแต่ละห้องมาเป็นแกน Y
9
10 avg_A1 = data.Var3;
11 avg_A2 = data.Var4;
12 avg_A3 = data.Var5;
13 Avg_A = data.Var6;
14
15 avg_B1 = data.Var8;
16 avg_B2 = data.Var9;
17 avg_B3 = data.Var10;
18 Avg_B = data.Var11;
19
20 avg_C1 = data.Var13;
21 avg_C2 = data.Var14;
22 avg_C3 = data.Var15;
23 Avg_C = data.Var16;
24
25 input_Voltage = 3.3;
26
27 % 3. สร้างหน้าต่างกราฟ
28 figure;
29
30 % 4. พล็อตกราฟเส้น
31
32 % plot(rotation, (avg_A1/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
33 % 6);
34 % hold on;
35 % plot(rotation, (avg_A2/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
36 % 6);
37 % plot(rotation, (avg_A3/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
38 % 6);
39 % plot(rotation, (Avg_A/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
40 % 6);
41 % plot(rotation, (avg_B1/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
42 % 6);
43 % plot(rotation, (avg_B2/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
44 % 6);
45 % plot(rotation, (avg_B3/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
46 % 6);
47 % plot(rotation, (Avg_B/input_Voltage) * 100, '-', 'LineWidth', 1.5, 'MarkerSize',
48 % 6);
49
50 % 5. ตกแต่งกราฟให้สวยงามและอ่านง่าย
51 title(sprintf('กราฟแสดงความสัมพันธ์ระหว่างระยะทางหมุน (Rotational Travel) และอัตราส่วนแรงดัน
52 (Output Voltage Ratio) ของ Rotary Potentiometer Type C'));
53 xlabel('Rotational Travel (%)');
54 ylabel('Output Voltage Ratio (%)');
55
56 % ใส่คำอธิบายเส้นกราฟ
57 % legend('Trial 1', 'Trial 2', 'Trial 3', 'Average Output Voltage Ratio A',
58 % 'Location', 'best');
59 % legend('Trial 1', 'Trial 2', 'Trial 3', 'Average Output Voltage Ratio B',
60 % 'Location', 'best');
61 % legend('Trial 1', 'Trial 2', 'Trial 3', 'Average Output Voltage Ratio C',
62 % 'Location', 'best');
```

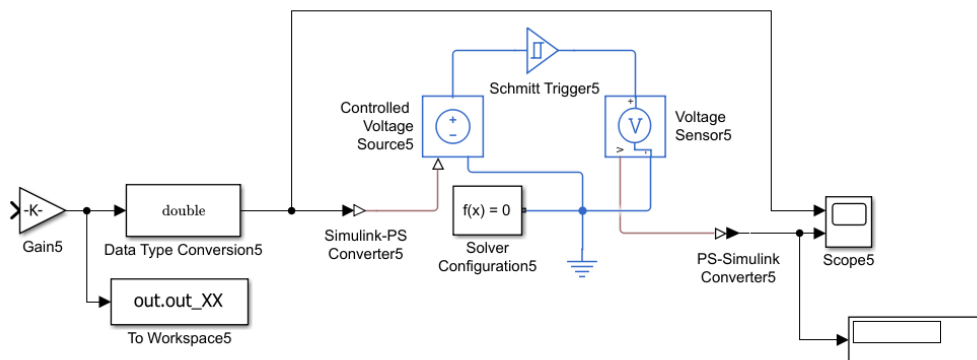
สคริปต์ในภาพนำข้อมูลจากไฟล์ Data_Lab1.1e.csv มาคำนวณหาอัตราส่วนแรงดันเอาต์พุต (Output Voltage Ratio %) เทียบกับแรงดันอินพุต 3.3V เพื่อพล็อตเปรียบเทียบคุณลักษณะ Taper Curve โดยครอบคลุมทั้ง Type A (Logarithmic), Type B (Linear) และ Type C (Inverse Logarithmic) ช่วยให้เห็นความสอดคล้องของผลการทดลองทั้ง 3 Trial เทียบกับกราฟมาตรฐานใน Datasheet ได้อย่างชัดเจน

1.2.3 สคริปต์สำหรับ Schmitt Trigger

```
1 % 1. อ่านข้อมูลจากไฟล์ CSV
2 data = readtable('Data_Lab1.1.csv');
3
4 % 2. กำหนดแกน X และแกน Y จากหัวคอลัมน์ในตาราง
5 % ใช้ Run_Number เป็นแกน X (Rotation)
6 time = data.Var18;
7
8 % ตั้งค่าเฉลี่ยของแต่ละช่องมาเป็นแกน Y
9
10 input_Voltage = data.Var19;
11 output_Voltage = data.Var20;
12 Threshold_High = 2;
13 Threshold_Low = 1;
14
15 % 3. สร้างหน้าต่างกราฟ
16 figure;
17
18 % 4. พล็อตกราฟเส้น
19
20 plot(time, input_Voltage, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
21 hold on;
22 plot(time, output_Voltage, '-', 'LineWidth', 1.5, 'MarkerSize', 6);
23
24 yline(Threshold_High, 'r--', 'LineWidth', 1.5, 'DisplayName', 'Threshold_High');
25 yline(Threshold_Low, 'b--', 'LineWidth', 1.5, 'DisplayName', 'Threshold_Low');
26
27 % 5. ตกแต่งกราฟให้สวยงามและอ่านง่าย
28 title('กราฟแสดงความสัมพันธ์ระหว่างแรงดัน Input (Voltage) และสัญญาณ Output (Schmitt Trigger)');
29 xlabel('Time (s)');
30 ylabel('Voltage (V)');
31
32 % ใส่คำอธิบายเส้นกราฟ
33 legend('Voltage (V)', 'Schmitt Trigger', 'Threshold_High', 'Threshold_Low', 'Location', 'best');
34
35 % เปิดตารางกริด
36 grid on;
37 hold off;
```

สคริปต์ในภาพทำหน้าที่ดึงข้อมูลรูปคลื่นสัญญาณรายเวลา (time) ของแรงดันอินพุต (input_Voltage) และสัญญาณดิจิทัลเอาต์พุต (output_Voltage) พร้อมสร้างเส้นระดับแรงดันเปรียบเทียบ Upper Threshold และ Lower Threshold ผ่านคำสั่ง yline() เพื่อแสดงการเปลี่ยนสถานะ Logic HIGH/LOW แบบ Real-time ตามช่วง Hysteresis Band ของวงจร Schmitt Trigger

1.3 แบบจำลอง Simulink และการวิเคราะห์ห้วงจร Schmitt Trigger



แบบจำลองในภาพทำหน้าที่รับสัญญาณแรงดันไฟฟ้าอนาล็อกจาก Potentiometer แล้วนำมาจำลองพฤติกรรมวงจร Schmitt Trigger ในโดเมนกายภาพ เพื่อสร้างสัญญาณดิจิทัลเอาต์พุตที่มีเสถียรภาพ โดยมีรายละเอียดการทำงานดังนี้

1. บล็อก Gain5: รับค่า ADC ดิจิทัล (0-4095) แล้วคูณด้วยปัจจัยขยาย ($\frac{3.3}{4095}$) เพื่อแปลงให้อยู่ในหน่วยแรงดันไฟฟ้า Volts (0–3.3V)
2. บล็อก Data Type Conversion5 (double): แปลงประเภทข้อมูลตัวเลขให้เป็นชนิด double เพื่อความแม่นยำในการคำนวณทางคณิตศาสตร์
3. บล็อก To Workspace5 (out.out_XX): ส่งข้อมูลแรงดันไฟฟ้าอนาล็อกเข้าสู่ MATLAB Workspace เพื่อให้ MATLAB Script นำไปหาค่าเฉลี่ย 10 วินาที และบันทึกลงไฟล์
4. บล็อก Simulink-PS Converter5: แปลงสัญญาณดิจิทัลจาก Simulink ให้กลายเป็นสัญญาณทางกายภาพ เพื่อป้อนเข้าสู่วงจรไฟฟ้าจำลอง
5. บล็อก Controlled Voltage Source5: ทำหน้าที่เป็นแหล่งจ่ายแรงดันไฟฟ้าอ้างอิง ที่สร้างแรงดันไฟฟ้าแปรผันตามค่าอนาล็อกอินพุต
6. บล็อก Schmitt Trigger5: บล็อกประมวลผลหลัก ทำหน้าที่เปรียบเทียบระดับแรงดันไฟฟ้าอินพุตกับขีดจำกัดแรงดัน 2 ระดับ (Hysteresis)
 - Upper Threshold (V_{TH}): 2.5V
 - Lower Threshold (V_{TL}): 1.0V

เมื่อแรงดันอินพุตสูงเกิน 2.5V สัญญาณเอาต์พุตจะเปลี่ยนเป็น HIGH (3.3V) และจะเปลี่ยนกลับเป็น LOW (0V) เมื่อแรงดันลดต่ำกว่า 1.0V โดยช่วง Hysteresis Band ขนาด 1.5V นี้ช่วยป้องกันการสลับสถานะไปมาอย่างรวดเร็ว จากสัญญาณรบกวน (Noise)
7. บล็อก Voltage Sensor5 & PS-Simulink Converter5: วัดระดับแรงดันดิจิทัลเอาต์พุตที่ได้จากวงจร Schmitt Trigger และแปลงสัญญาณกายภาพกลับมาเป็นสัญญาณ Simulink
8. บล็อก Scope5: แสดงผลเปรียบเทียบรูปคลื่นสัญญาณอนาล็อกอินพุตกับสัญญาณดิจิทัลเอาต์พุตแบบ Real-time

2. Lab 1.2 Magnetic Sensor

2.1 โปรแกรมรัน Simulink 10 วินาที และบันทึกข้อมูลดิบพร้อมค่าเฉลี่ยลงไฟล์ .csv

ในการทดลองใช้ MATLAB Script ในการควบคุมการรัน Simulink Model (Simulink_Lab1.4) เป็นเวลา 10 วินาที ณ แต่ละระยะทดสอบ โดยทำการอ่านค่าแรงดันอนาล็อก (A0) และค่าดิจิทัลดิบ (Raw Bit) พร้อมคำนวณค่าเฉลี่ยสะสมเพื่อลดสัญญาณรบกวน แล้วบันทึกออกเป็นไฟล์ โดยสคริปต์จะทำการดึงข้อมูล Time-series Raw Data และคำนวณค่าแรงดันไฟฟ้าเฉลี่ย (Sample Averaging) เพื่อลดสัญญาณรบกวน (Noise Reduction) พร้อมบันทึกข้อมูลลงไฟล์ .csv 2 รูปแบบ คือ ไฟล์ตารางค่าเฉลี่ยสะสม (Hall_Effect_Average.csv) และ ไฟล์ตารางข้อมูลดิบแยกรายรอบ (Hall_Effect_Average_[N/NS/S/SS]_Run%d.csv)

```
1 % 1. โหลดโมเดล
2 model_name = 'Simulink_Lab1.2';
3 load_system(model_name);
4
5 % 2. ตรวจสอบและสร้างตัวแปร
6 if ~exist('History_Avg_Run', 'var')
7     % ตัวแปรเก็บค่าเฉลี่ย
8     History_Avg_Run = [];
9     History_Avg_A0 = [];
10    History_Avg_Raw = [];
11    Run_Count = 0;
12 end
13
14 disp('--- โปรแกรมเก็บข้อมูล A0 และ Raw (ส่งออก CSV: ค่าเฉลี่ย 1 ไฟล์ + Raw Data แยกตามรอบ) ---');
15
16 % 3. เริ่มวนลูป
17 while true
18     str = input('\nกด Enter รัน 10 วิ (พิมพ์ c เพื่อล้างค่า, พิมพ์ q เพื่อออก): ', 's');
19
20     if strcmpi(str, 'q')
21         clear History_Avg_Run History_Avg_A0 History_Avg_Raw Run_Count;
22         disp('❏ ออกจากโปรแกรม และล้างค่าเรียบร้อยแล้ว!');
23         break;
24
25     elseif strcmpi(str, 'c')
26         History_Avg_Run = []; History_Avg_A0 = []; History_Avg_Raw = [];
27         Run_Count = 0;
28         disp('❏ ล้างข้อมูลเก่าออกหมดแล้ว! พร้อมรันรอบ 1 ใหม่!');
29         continue;
30     end
31
32     Run_Count = Run_Count + 1;
33     fprintf('\n🔧 กำลังรัน Simulink รอบที่ %d...\n', Run_Count);
34
35     % สั่งรัน 10 วินาที
36     out = sim(model_name, 'StopTime', '10');
37
38     % จัดการ Raw Data ---
39     % ดึงค่าจากบล็อก out_A1 และ out_B1 ใน Simulink
40     time_sec = out.out_A0.Time;
41     A0_data = out.out_A0.Data;
42     Raw_data = out.out_Raw.Data;
43
44     % --- จัดการ Average Data ---
45     cur_mean_A0 = mean(A0_data);
46     cur_mean_Raw = mean(Raw_data);
47
48     % สะสมค่าเฉลี่ย
49     History_Avg_Run(end+1, 1) = Run_Count;
50     History_Avg_A0(end+1, 1) = cur_mean_A0;
51     History_Avg_Raw(end+1, 1) = cur_mean_Raw;
52
53     fprintf('❏ ค่าเฉลี่ยรอบที่ %d → A1: %.4f mV | B1: %.4f Bit\n', ...
54         Run_Count, cur_mean_A0, cur_mean_Raw);
55
56     % =====
57     % 4. บันทึกเป็นไฟล์ CSV (แยกตามรอบ)
58     % =====
59
60     % 4.1 ไฟล์ที่ 1: ตารางค่าเฉลี่ย (อัปเดตไฟล์เดิม)
61     T_avg = table(History_Avg_Run, History_Avg_A0, History_Avg_Raw, ...
62         'VariableNames', {'Run_Number', 'Avg_Voltage', 'Avg_Bit'});
63     writetable(T_avg, 'Hall_Effect_Average.csv');
64
65     % 4.2 ไฟล์ที่ 2, 3, 4... : ตารางข้อมูลดิบ (สร้างไฟล์ใหม่รายรอบ)
66     T_raw_current = table(time_sec, A0_data, Raw_data, ...
67         'VariableNames', {'Time_sec', 'Voltage', 'Bit'});
68
69     % ตั้งชื่อไฟล์ให้ตรงกับหมายเลขรอบ เช่น Potentiometer_RawData_Run1.csv
70     filename_raw = sprintf('Hall_Effect_Average_S_Run%d.csv', Run_Count);
71     writetable(T_raw_current, filename_raw);
72
73     fprintf('❏ อัปเดตไฟล์สำเร็จ! (ได้ไฟล์ %s)\n', filename_raw);
74 end
```


2.2 สคริปต์สำหรับการโหลดไฟล์ csv และพล็อตแสดงผลกราฟ

สคริปต์นี้ทำหน้าที่นำข้อมูลแรงดันไฟฟ้าจาก Data_Lab1.2.csv มาคำนวณหาค่าความหนาแน่นฟลักซ์แม่เหล็ก (Magnetic Flux Density: B ในหน่วย mT) และพล็อตเปรียบเทียบระหว่างสภาวะมีแผ่นกำบัง (Shield) กับไม่มีแผ่นกำบัง (No Shield) ทั้งขั้วเหนือ (North) และขั้วใต้ (South)

สูตรคำนวณที่ใช้ในสคริปต์

- Quiescent Voltage $\{V\}_{Q}$: 2500 mV (สำหรับ $V_{cc}=5V$)
- Sensitivity: 12.5 mV/ mT
- Magnetic Flux Density (B):

$$B = \frac{(V_{avg} - V_Q)}{\text{Sensitivity}} \quad [mT]$$

```
1 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
2 % LAB 1.2: Magnetic Sensor Data Plotting Script
3 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
4
5 % 1. อ่านข้อมูลจากไฟล์ CSV
6 data = readtable('Data_Lab1.2.csv');
7
8 % 2. กำหนดค่าพารามิเตอร์และข้อมูลจากตาราง
9 V_Q = 2500; % Quiescent Voltage (mV) สำหรับ Vcc = 5V
10 Sensitivity = 12.5; % Sensitivity (mV/mT)
11
12 distance = data.distance;
13
14 % คำนวณค่าแรงดันไฟฟ้า (Voltage Average: mV)
15
16 % North - Shield %
17 Avg_NS_1 = data.Avg_NS_1 ;
18 Avg_NS_2 = data.Avg_NS_2 ;
19 Avg_NS_3 = data.Avg_NS_3 ;
20 Avg_NS = data.Avg_NS ;
21 % North - Shield %
22
23 % North - No Shield %
24 Avg_N_1 = data.Avg_N_1 ;
25 Avg_N_2 = data.Avg_N_2 ;
26 Avg_N_3 = data.Avg_N_3 ;
27 Avg_N = data.Avg_N ;
28 % North - No Shield %
29
30 % South - Shield %
31 Avg_SS_1 = data.Avg_SS_1 ;
32 Avg_SS_2 = data.Avg_SS_2 ;
33 Avg_SS_3 = data.Avg_SS_3 ;
34 Avg_SS = data.Avg_SS ;
35 % South - Shield %
36
37 % South - No Shield %
38 Avg_S_1 = data.Avg_S_1 ;
39 Avg_S_2 = data.Avg_S_2 ;
40 Avg_S_3 = data.Avg_S_3 ;
41 Avg_S = data.Avg_S ;
42 % South - No Shield %
43
44 % คำนวณค่าความหนาแน่นฟลักซ์แม่เหล็ก (Magnetic Flux Density: B ในหน่วย mT)
45
46 % North - Shield %
47 B_NS_1 = ( Avg_NS_1 - V_Q ) / Sensitivity ;
48 B_NS_2 = ( Avg_NS_2 - V_Q ) / Sensitivity ;
49 B_NS_3 = ( Avg_NS_3 - V_Q ) / Sensitivity ;
50 B_NS = ( Avg_NS - V_Q ) / Sensitivity ;
51 % North - Shield %
52
53 % North - No Shield %
54 B_N_1 = ( Avg_N_1 - V_Q ) / Sensitivity ;
55 B_N_2 = ( Avg_N_2 - V_Q ) / Sensitivity ;
56 B_N_3 = ( Avg_N_3 - V_Q ) / Sensitivity ;
57 B_N = ( Avg_N - V_Q ) / Sensitivity ;
58 % North - No Shield %
59
60 % South - Shield %
61 B_SS_1 = ( Avg_SS_1 - V_Q ) / Sensitivity ;
62 B_SS_2 = ( Avg_SS_2 - V_Q ) / Sensitivity ;
63 B_SS_3 = ( Avg_SS_3 - V_Q ) / Sensitivity ;
64 B_SS = ( Avg_SS - V_Q ) / Sensitivity ;
65 % South - Shield %
66
67 % South - No Shield %
68 B_S_1 = ( Avg_S_1 - V_Q ) / Sensitivity ;
69 B_S_2 = ( Avg_S_2 - V_Q ) / Sensitivity ;
70 B_S_3 = ( Avg_S_3 - V_Q ) / Sensitivity ;
71 B_S = ( Avg_S - V_Q ) / Sensitivity ;
72 % South - No Shield %
73
74 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
75 % รูปที่ 1: กราฟเปรียบเทียบ Average North - Shield
76 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
77
78 fig1 = figure(1);
79 set(fig1, 'Name', 'Average North - Shield', 'NumberTitle', 'off');
80
81 plot(distance, Avg_NS_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
82 'Trial 1');
83 hold on;
84 plot(distance, Avg_NS_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
85 'Trial 2');
86 plot(distance, Avg_NS_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
87 'Trial 3');
88 plot(distance, Avg_NS, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
89 'Average North - Shield');
90 hold off;
91
92 title('กราฟแสดงความสัมพันธ์ระหว่างระยะทาง (Distance) และแรงดันไฟฟ้า (Voltage)');
93 xlabel('Distance (cm)');
94 ylabel('Voltage (mV)');
95 legend('Location', 'best');
96 grid on;
97
98 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
99 % รูปที่ 2: กราฟเปรียบเทียบ Average North - No Shield
100 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
101
102 fig2 = figure(2);
103 set(fig2, 'Name', 'Average North - No Shield', 'NumberTitle', 'off');
104
105 plot(distance, Avg_N_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
106 'Trial 1');
107 hold on;
108 plot(distance, Avg_N_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
109 'Trial 2');
110 plot(distance, Avg_N_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
111 'Trial 3');
112 plot(distance, Avg_N, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
113 'Average North - No Shield');
114 hold off;
115
116 title('กราฟแสดงความสัมพันธ์ระหว่างระยะทาง (Distance) และแรงดันไฟฟ้า (Voltage)');
117 xlabel('Distance (cm)');
118 ylabel('Voltage (mV)');
119 legend('Location', 'best');
120 grid on;
121
122 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
123 % รูปที่ 3: กราฟเปรียบเทียบ Average South - Shield
124 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
125
126 fig3 = figure(3);
127 set(fig3, 'Name', 'Average South - Shield', 'NumberTitle', 'off');
128
129 plot(distance, Avg_SS_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
130 'Trial 1');
131 hold on;
132 plot(distance, Avg_SS_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
133 'Trial 2');
134 plot(distance, Avg_SS_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
135 'Trial 3');
136 plot(distance, Avg_SS, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
137 'Average South - Shield');
138 hold off;
139
140 title('กราฟแสดงความสัมพันธ์ระหว่างระยะทาง (Distance) และแรงดันไฟฟ้า (Voltage)');
141 xlabel('Distance (cm)');
142 ylabel('Voltage (mV)');
143 legend('Location', 'best');
144 grid on;
```

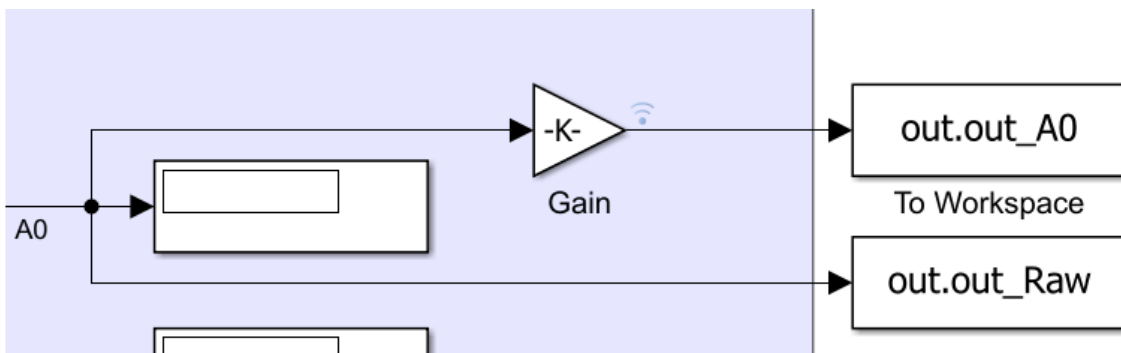


```

1 % =====
2 % รูปที่ 4: กราฟแสดงค่า Average South - No Shield
3 % =====
4
5 fig4 = figure(4);
6 set(fig4, 'Name', 'Average South - No Shield', 'NumberTitle', 'off');
7
8 plot(distance, Avg_S_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
9 'Trial 1');
9 hold on;
10 plot(distance, Avg_S_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
11 'Trial 2');
11 plot(distance, Avg_S_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
12 'Trial 3');
12 plot(distance, Avg_S, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
13 'Average South - No Shield');
13 hold off;
14
15 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และแรงดันไฟฟ้า (Voltage)');
16 xlabel('Distance (cm)');
17 ylabel('Voltage (mV)');
18 legend('Location', 'best');
19 grid on;
20
21 % =====
22 % รูปที่ 5: กราฟแสดงค่า Voltage vs Distance (กรณีไม่มี Shielding)
23 % =====
24 fig5 = figure(5);
25 set(fig5, 'Name', 'Distance vs Voltage', 'NumberTitle', 'off');
26
27 plot(distance, Avg_N, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
28 'North - No Shield');
28 hold on;
29 plot(distance, Avg_NS, '-s', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
30 'North - Shield');
30 plot(distance, Avg_S, '-+', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
31 'South - No Shield');
31 plot(distance, Avg_SS, '-v', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
32 'South - Shield');
32 hold off;
33
34 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และแรงดันไฟฟ้า (Voltage)');
35 xlabel('Distance (cm)');
36 ylabel('Voltage (mV)');
37 legend('Location', 'best');
38 grid on;
39
40 % =====
41 % รูปที่ 6: กราฟแสดงค่า B North - Shield
42 % =====
43
44 fig6 = figure(6);
45 set(fig6, 'Name', 'B North - Shield', 'NumberTitle', 'off');
46
47 plot(distance, Avg_NS_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
48 'Trial 1');
48 hold on;
49 plot(distance, Avg_NS_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
50 'Trial 2');
50 plot(distance, Avg_NS_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
51 'Trial 3');
51 plot(distance, Avg_NS, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'B
52 North - Shield');
52 hold off;
53
54 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และความหนาแน่นสนามแม่เหล็ก (B)');
55 xlabel('Distance (cm)');
56 ylabel('Magnetic Field Density (mT)');
57 legend('Location', 'best');
58 grid on;
59
60 % =====
61 % รูปที่ 7: กราฟแสดงค่า B North - No Shield
62 % =====
63
64 fig7 = figure(7);
65 set(fig7, 'Name', 'B North - No Shield', 'NumberTitle', 'off');
66
67 plot(distance, B_N_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
68 'Trial 1');
68 hold on;
69 plot(distance, B_N_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Trial
70 2');
70 plot(distance, B_N_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
71 'Trial 3');
71 plot(distance, B_N, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'B
72 North - No Shield');
72 hold off;
73
74 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และความหนาแน่นสนามแม่เหล็ก (B)');
75 xlabel('Distance (cm)');
76 ylabel('Magnetic Field Density (mT)');
77 legend('Location', 'best');
78 grid on;
79
80 % =====
81 % รูปที่ 8: กราฟแสดงค่า B South - Shield
82 % =====
83
84 fig8 = figure(8);
85 set(fig8, 'Name', 'B South - Shield', 'NumberTitle', 'off');
86
87 plot(distance, B_SS_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
88 'Trial 1');
88 hold on;
89 plot(distance, B_SS_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
90 'Trial 2');
90 plot(distance, B_SS_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
91 'Trial 3');
91 plot(distance, B_SS, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'B
92 South - Shield');
92 hold off;
93
94 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และความหนาแน่นสนามแม่เหล็ก (B)');
95 xlabel('Distance (cm)');
96 ylabel('Magnetic Field Density (mT)');
97 legend('Location', 'best');
98 grid on;
99
100 % =====
101 % รูปที่ 9: กราฟแสดงค่า B South - No Shield
102 % =====
103
104 fig9 = figure(9);
105 set(fig9, 'Name', 'B South - No Shield', 'NumberTitle', 'off');
106
107 plot(distance, B_S_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
108 'Trial 1');
108 hold on;
109 plot(distance, B_S_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Trial
110 2');
110 plot(distance, B_S_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
111 'Trial 3');
111 plot(distance, B_S, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'B
112 South - No Shield');
112 hold off;
113
114 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และความหนาแน่นสนามแม่เหล็ก (B)');
115 xlabel('Distance (cm)');
116 ylabel('Magnetic Field Density (mT)');
117 legend('Location', 'best');
118 grid on;
119
120 % =====
121 % รูปที่ 10: กราฟแสดงค่า Magnetic Flux Density (B) vs Distance
122 % =====
123 fig10 = figure(10);
124 set(fig10, 'Name', 'Magnetic Flux Density (B) vs Distance', 'NumberTitle', 'off');
125
126 plot(distance, B_N, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'North
127 - No Shield');
127 hold on;
128 plot(distance, B_NS, '-s', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
129 'North - Shield');
129 plot(distance, B_S, '-+', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'South
130 - No Shield');
130 plot(distance, B_SS, '-v', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
131 'South - Shield');
131 hold off;
132
133 title('กราฟแสดงค่าเฉลี่ยของระยะทาง (Distance) และความหนาแน่นสนามแม่เหล็ก (B)');
134 xlabel('Distance (cm)');
135 ylabel('Magnetic Field Density (mT)');
136 legend('Location', 'best');
137 grid on;
138
139 % =====
140 % รูปที่ 11: กราฟแสดงค่าความสัมพันธ์ North Pole (Linearity Curve: Voltage vs B)
141 % =====
142 fig11 = figure(11);
143 set(fig11, 'Name', 'Voltage vs B (North Pole)', 'NumberTitle', 'off');
144
145 plot(B_N, Avg_N, 'bo', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'North
146 Pole (No Shield)');
146 hold on;
147 plot(B_NS, Avg_NS, 's', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'North
148 Pole (Shield)');
148 hold off;
149
150 title('กราฟแสดงค่าความสัมพันธ์ของค่าแรงดันไฟฟ้ากับค่าความหนาแน่นสนามแม่เหล็ก (Sensor Linearity)');
151 xlabel('Magnetic field Density (mT)');
152 ylabel('Voltage (mV)');
153 legend('Location', 'northwest');
154 grid on;
155
156 % =====
157 % รูปที่ 12: กราฟแสดงค่าความสัมพันธ์ South Pole (Linearity Curve: Voltage vs B)
158 % =====
159 fig12 = figure(12);
160 set(fig12, 'Name', 'Voltage vs B (South Pole)', 'NumberTitle', 'off');
161
162 plot(B_S, Avg_S, 'bo', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'South
163 Pole (No Shield)');
163 hold on;
164 plot(B_SS, Avg_SS, 's', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'South
165 Pole (Shield)');
165 hold off;
166
167 title('กราฟแสดงค่าความสัมพันธ์ของค่าแรงดันไฟฟ้ากับค่าความหนาแน่นสนามแม่เหล็ก (Sensor Linearity)');
168 xlabel('Magnetic Field Density (mT)');
169 ylabel('Voltage (mV)');
170 legend('Location', 'northwest');
171 grid on;

```

2.3 แบบจำลอง Simulink

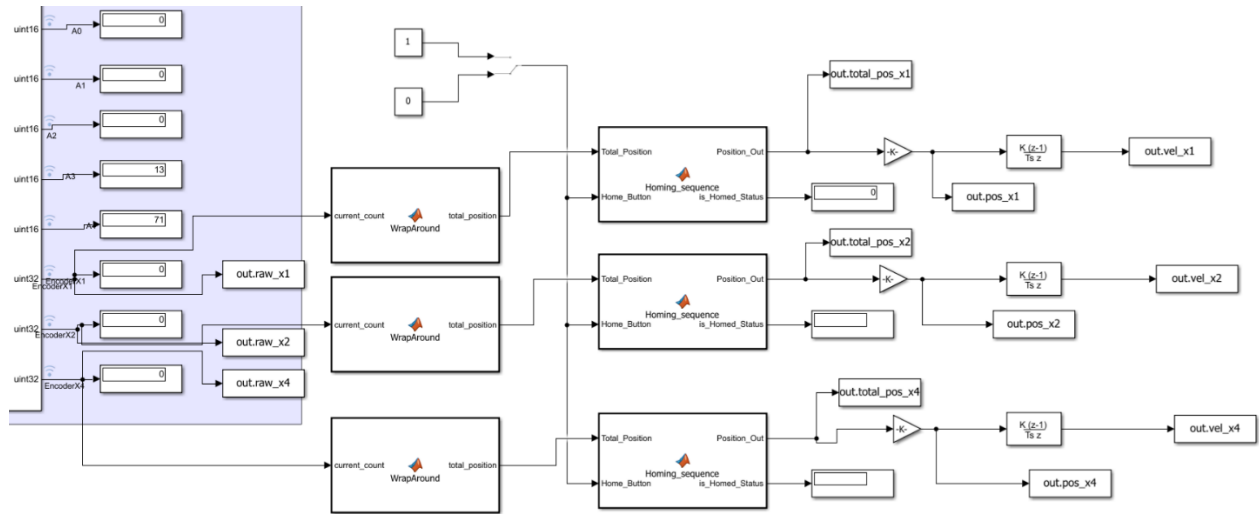


แบบจำลองในภาพที่ ข.3 รับสัญญาณแรงดันไฟฟ้าอนาล็อกจากเซนเซอร์วัดสนามแม่เหล็ก DRV5055 ผ่านพอร์ต Analog Input (A0) เพื่ออ่านค่านับดิจิทัลและแปลงเป็นค่าแรงดันไฟฟ้าเอาต์พุต (V_{out}) ก่อนส่งออกไปยัง MATLAB Workspace สำหรับนำไปคำนวณหาค่าความเข้มสนามแม่เหล็ก (B) ต่อไป โดยมีรายละเอียดการทำงานของแต่ละบล็อกดังนี้

1. บล็อก Gain: รับค่า ADC ดิจิทัล (0-4095) แล้วคูณด้วยปัจจัยขยาย ($\frac{3.3}{4095}$) เพื่อแปลงให้อยู่ในหน่วยแรงดันไฟฟ้า Volts (0-3.3V)
2. บล็อก out.out_A0 (To Workspace): ส่งข้อมูลแรงดันไฟฟ้าเอาต์พุต (V_{out}) เข้าสู่ MATLAB Workspace เพื่อให้ MATLAB Script นำไปคำนวณหาค่าความเข้มสนามแม่เหล็ก (Magnetic Flux Density: B) ในหน่วย mT ผ่านสมการ และบันทึกผลการทดลองลงไฟล์ .csv ต่อไป
3. บล็อก out.out_Raw (To Workspace): ส่งข้อมูลค่านับดิจิทัลดิบ (Raw ADC Bits) เข้าสู่ MATLAB Workspace เพื่อใช้ในการตรวจสอบและวิเคราะห์สัญญาณรบกวน (Noise Analysis)

3. Lab 1.3 Incremental Encoder

3.1 การออกแบบโปรแกรมสำหรับ Simulink



3.2 การออกแบบโปรแกรม WrapAround



```
function total_position = WrapAround(current_count)

% 1. แปลงค่า input ให้เป็น double
current_count = double(current_count);

% *** ค่าสูงสุดของ 16-bit Timer ***
MAX_COUNT = 65535;

% ตั้งค่า Threshold ไว้ประมาณครึ่งหนึ่ง
THRESHOLD = 30000;

persistent prev_count;
persistent accumulated_pos;

if isempty(prev_count)
    prev_count = current_count;
    accumulated_pos = 0;
end

delta = current_count - prev_count;

if delta < -THRESHOLD
    delta = delta + (MAX_COUNT + 1);
elseif delta > THRESHOLD
    delta = delta - (MAX_COUNT + 1);
end

accumulated_pos = accumulated_pos + delta;
prev_count = current_count;
total_position = accumulated_pos;
end
```

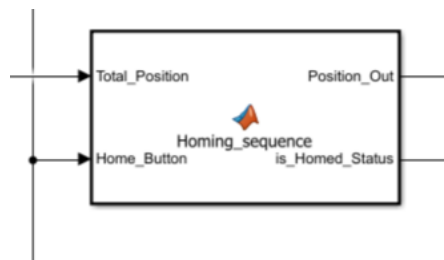
การออกแบบโปรแกรม Wraparound เพื่อลดปัญหาค่ากระโดดไปอีกฝั่งเมื่อมีการนับน้อยกว่า 0 หรือมากกว่า 65535 เนื่องจาก 16-bit Timer จะรับค่าได้แค่ 0 ถึง 65535

การคำนวณความต่าง โดยโปรแกรมจะนำค่าปัจจุบันลบด้วยค่าครั้งก่อน

ตรวจจับ Overflow: เมื่อค่าความต่างเป็น ค่าลบ ขนาดใหญ่ (หมุนทะลุ 65535 วนกลับไป 0) จะทำให้เข้าเงื่อนไข $\text{delta} < -\text{THRESHOLD}$ และทำการคำนวณแก้ไขด้วยสมการ $\text{delta} = \text{delta} + (\text{MAX_COUNT} + 1)$

ตรวจจับ Underflow: เมื่อค่าความต่างเป็น ค่าบวก ขนาดใหญ่ (หมุนย้อนจาก 0 วนกลับไป 65535) จะทำให้เข้าเงื่อนไข $\text{delta} > \text{THRESHOLD}$ และทำการคำนวณแก้ไขด้วยสมการ $\text{delta} = \text{delta} - (\text{MAX_COUNT} + 1)$ จากนั้นโปรแกรมจะนำค่า delta ที่ผ่านการแก้ไขแล้ว ไปบวกสะสมอย่างต่อเนื่องเพื่อให้ได้ตำแหน่งที่ถูกต้อง

3.3 การออกแบบโปรแกรม Homing sequence



```
function [Position_Out, is_Homed_Status] = Homing_sequence(Total_Position, Home_Button)
% ไม่ persistent เพื่อจำค่าออฟเซตและสถานะข้ามรอบการทำงาน (Sample Time)
persistent home_offset;
persistent homed_flag;

% กำหนดค่าเริ่มต้นเมื่อรันโปรแกรม
if isempty(homed_flag)
    homed_flag = false;
    home_offset = 0;
end

% ถ้ามีการกดปุ่ม (รับค่าลอจิกเป็น 1) ให้เซตตำแหน่งปัจจุบันเป็น 0
if (Home_Button == 1)
    home_offset = Total_Position; % บันทึกตำแหน่งพัลส์สะสม ณ ตอนที่กดปุ่ม
    homed_flag = true;           % เปลี่ยนสถานะว่าทำการโฮมแล้ว
end

% คำนวณตำแหน่งสุทธิ (หักลบจุดอ้างอิง)
Position_Out = Total_Position - home_offset;
is_Homed_Status = homed_flag;
end
```

การกำหนดตัวแปรเริ่มต้น: โปรแกรมใช้ตัวแปรประเภท persistent ที่สามารถจำค่าก่อนหน้าได้ เพื่อจดจำค่า offset และสถานะให้คงอยู่ข้ามรอบการทำงานโดยเมื่อเริ่มต้นโปรแกรมจะตั้งสถานะการโฮมเป็น false และค่าออฟเซตเป็น 0 การเซ็ต Home : เมื่อมีการกดปุ่ม ซึ่งรับค่าลอจิกอินพุตเป็น 1 ส่งผลให้ Home_Button = 1 โปรแกรมจะทำการบันทึก Total_Position ณ ตอนที่กดปุ่ม เก็บไว้เป็นค่า offset พร้อมทั้งเปลี่ยนสถานะว่าทำการโฮมแล้วเป็น true

การคำนวณตำแหน่งสุทธิ: โปรแกรมจะคำนวณตำแหน่งใหม่ที่หักลบจุดอ้างอิงแล้วตามโค้ด $Position_Out = Total_Position - home_offset$ และส่งออกค่าสถานะการโฮม $is_Homed_Status = homed_flag$ เพื่อให้ได้ตำแหน่งที่เริ่มต้นจาก 0 จริง ๆ ณ ตำแหน่งที่ผู้ใช้กดปุ่ม

4. Lab 1.4 Load Cell

4.1 โปรแกรมรัน Simulink และบันทึกข้อมูลดิบพร้อมค่าเฉลี่ยลงไฟล์ .csv

ในการทดลองใช้ MATLAB Script ในการควบคุมการรัน Simulink Model (Simulink_Lab1.4) ในการควบคุมกระบวนการเก็บข้อมูลและการปรับเทียบมาตรฐาน (Calibration) ร่วมกับแบบจำลอง Simulink (Loadcell) โดยมีรายละเอียดการทำงานดังนี้

```
1 % =====
2 % คลังโปรแกรม Calibration ที่จัดไว้ให้ (ระบบ Set Zero / Tare)
3 % =====
4
5 % 1. โหลดค่าเริ่มต้นเข้ามาจากไฟล์ 'น้ำหนัก.csv'
6 try
7     T = readtable('น้ำหนัก.csv');
8     % สืบข้อมูลจากคอลัมน์ที่ 3
9     target_weights = T(:, 3);
10 catch
11     disp('⚠️ โหลดไฟล์ น้ำหนัก.csv ไม่พบไฟล์นี้! กรุณาใส่ค่าเริ่มต้น 0, 50, 100... แทน');
12     target_weights = [0; 50; 100; 150; 200; 250; 300];
13 end
14
15 % 2. เตรียมตัวรัน
16 weight_array = [];
17 volt_array = [];
18 current_step = 1;
19 v_zero_offset = 0; % สำหรับค่าปรับเทียบ Set Zero เป็นครั้งที่ 0
20
21 % *** เมื่อไฟล์ Simulink ***
22 model_name = 'loadcell';
23
24 disp('=== เริ่มโปรแกรมเก็บข้อมูล Calibration สำหรับ Load Cell ===');
25
26 % 3. เริ่มการบันทึกค่าที่รันอยู่
27 while true
28     fprintf('\n-----\n');
29     if current_step <= length(target_weights)
30         current_w = target_weights(current_step);
31         fprintf('📌 เริ่มน้ำหนัก: กรุณารอจนหนัก [ Xd กรัม ] ลงบนเซ็นเซอร์', current_w);
32     else
33         current_w = NaN;
34         fprintf('📌 กรุณาการวัดความหนักค่าที่บันทึกในตารางแล้ว!\n');
35     end
36
37 % เตรียม [Z] ในหน่วยกรัม
38 fprintf('[Z] เช็ดถาด (Tare) | [J] เบ้า 50 กรัม | [K] ลงน้ำหนัก | [C] ค่าค่าทั้งหมด | [L] ออกและรีเซ็ต\n');
39
40 cmd = input('พิมพ์คำสั่งแล้วกด Enter: ', 's');
41 cmd = upper(strtrim(cmd));
42
43 if strcmp(cmd, 'Z')
44     % ----- ระบบ Set Zero -----
45     disp('📌 ทำ Simulink เพื่อเช็ดถาด... (กรุณาลงน้ำหนักจากเซ็นเซอร์ให้หมด)');
46     try
47         out = sim(model_name);
48         y_all = out.y_volt;
49
50         if length(y_all) >= 50
51             y_50_points = y_all(end-49:end);
52         else
53             y_50_points = y_all;
54         end
55
56 % บันทึกค่าเฉลี่ยเป็นค่า Offset
57 v_zero_offset = mean(y_50_points);
58 fprintf('📌 เช็ดถาดสำเร็จ! ค่าเฉลี่ยปัจจุบันคือ %.4f โวลต์\n', v_zero_offset);
59
60 catch ME
61     disp('❌ รัน Simulink ไม่สำเร็จ! กรุณาเช็คชื่อไฟล์โมเดลหรือการเชื่อมต่อระบบ');
62     disp(ME.message);
63 end
64
65 elseif strcmp(cmd, 'J')
66     % ----- ระบบเบ้า -----
67     if isempty(current_w)
68         disp('⚠️ น้ำหนักเป็นศูนย์! กรุณาใส่ค่า L เพื่อออกคณ');
69         continue;
70     end
71     disp('📌 ทำ Simulink เพื่อเก็บข้อมูล 50 จุด... (น้ำหนักคงที่)');
72
73     try
74         out = sim(model_name);
75
76         y_all = out.y_volt;
77         if length(y_all) >= 50
78             y_50_points = y_all(end-49:end);
79         else
80             y_50_points = y_all;
81         end
82
83 % นำค่าที่คำนวณได้ มาลบกับค่า Zero Offset
84 avg_v = mean(y_50_points) - v_zero_offset;
85
86 weight_array = [weight_array; current_w];
87 volt_array = [volt_array; avg_v];
88
89 fprintf('✓ บันทึกสำเร็จ: น้ำหนัก %.4f กรัม = %.4f โวลต์ (หัก Zero แล้ว)\n',
90         current_w, avg_v);
91
92 current_step = current_step + 1;
93
94 catch ME
95     disp('❌ รัน Simulink ไม่สำเร็จ!');
96     disp(ME.message);
97 end
98
99 elseif strcmp(cmd, 'K')
100     if isempty(weight_array)
101         fprintf('📌 กรุณาใส่ค่า น้ำหนัก Xd กรัม, แรงดัน %.4f V ออกแล้ว!\n',
102             weight_array(end), volt_array(end));
103         weight_array(end) = [];
104         volt_array(end) = [];
105         current_step = 1;
106     else
107         disp('⚠️ ยังไม่มีข้อมูลในระบบให้ลบคณ');
108     end
109
110 elseif strcmp(cmd, 'C')
111     weight_array = [];
112     volt_array = [];
113     current_step = 1;
114     v_zero_offset = 0; % รีเซ็ตค่า Zero คืนค่าเป็นค่า
115     disp('📌 ค่าค่าทั้งหมด (รวมถึงค่า Zero) รีเซ็ตแล้ว โปรแกรมกลับไปเริ่มต้นใหม่');
116
117 elseif strcmp(cmd, 'L')
118     disp('📌 ทำ Simulink บันทึกและจัดเก็บค่าเรียบร้อยแล้ว...');
119     break;
120
121 else
122     disp('❌ คำสั่งไม่ถูกต้อง! พิมพ์ Z, J, K, C หรือ L');
123 end
124 end
```

```
1 % =====
2 % 5. บันทึกไฟล์ Excel (.xlsx) ลงรวม Path และบันทึกไฟล์นี้ไว้
3 % =====
4 if ~isempty(weight_array)
5     ResultTable = table(weight_array, volt_array, 'VariableNames', {'Weight_g',
6         'Adjusted_Voltage_V'});
7
8 % 5.1 กำหนด Path ที่ต้องการเซฟไฟล์
9 save_path = 'D:\kmute\FRA273_RMX\LAB1.4\load cell';
10
11 if ~exist(save_path, 'dir')
12     mkdir(save_path);
13 end
14
15 % 5.2 ตั้งชื่อไฟล์ตามที่ต้องการ
16 base_filename = 'Calibration_Result';
17 ext = '.xlsx';
18 filename = [base_filename, ext];
19 full_filepath = fullfile(save_path, filename);
20
21 % 5.3 เซฟไฟล์ค่า (ค่าที่ใส่คือข้อมูลค่า) ให้เป็นเลขต่อท้ายเป็น _1, _2, _3)
22 counter = 1;
23 while exist(fullfile(save_path, filename), 'file')
24     filename = sprintf('%s_xdxe', base_filename, counter, ext);
25     full_filepath = fullfile(save_path, filename);
26     counter = counter + 1;
27 end
28
29 % 5.4 บันทึกไฟล์ Excel ตามชื่อที่ต้องการเซฟแล้ว
30 writetable(ResultTable, full_filepath);
31 fprintf('📌 บันทึกข้อมูลสำเร็จ! สามารถเข้าไปดูไฟล์ได้ที่: %s\n', full_filepath);
32
33 figure('Name', 'Calibration Curve', 'NumberTitle', 'off');
34 plot(weight_array, volt_array, '-ob', 'LineWidth', 1.5, 'MarkerSize', 8,
35     'MarkerFaceColor', 'b');
36 title('Sensor Calibration Curve (Tare Applied)');
37 xlabel('Weight (grams)');
38 ylabel('Adjusted Voltage (V)');
39 grid on;
40
41 if length(weight_array) > 1
42     p = polyfit(weight_array, volt_array, 1);
43     eq_text = sprintf('สมการ Calibration: \n Voltage = (%.5f * Weight) + %.5f',
44         p(1), p(2));
45     text(min(weight_array), max(volt_array), eq_text, 'VerticalAlignment',
46         'top', 'FontSize', 10, 'Color', 'r', 'FontWeight', 'bold');
47 else
48     disp('⚠️ มีข้อมูลแค่ 1 จุด จึงไม่สามารถคำนวณหาสมการเส้นตรง (Calibration Equation) ได้');
49 end
50
51 disp('⚠️ ไม่มีข้อมูลถูกบันทึก จึงไม่มีการสร้างไฟล์และกราฟคณ');
52 end
```

สคริปต์นี้ถูกออกแบบมาเพื่อควบคุมกระบวนการเก็บสัญญาณออกจาก INA125 คำนวณค่าเฉลี่ย เซ็ตศูนย์ (Tare) และจัดทำ Calibration Curve โดยมีขั้นตอนการทำงานดังนี้

1. การโหลดและตั้งค่าพารามิเตอร์เริ่มต้น (บรรทัดที่ 5–22): สคริปต์เริ่มด้วยการอ่านไฟล์น้ำหนัก.csv เพื่อโหลดค่าน้ำหนักเป้าหมาย (หากไม่พบไฟล์จะใช้ค่าเริ่มต้น [0; 50; 100; ...; 300]) กำหนดตัวแปรสำหรับเก็บค่าน้ำหนัก แรงดันไฟฟ้า Offset และระบุชื่อแบบจำลอง Simulink (model_name = 'Loadcell')
2. การควบคุมลูปและการรับคำสั่งจากผู้ใช้ (บรรทัดที่ 27–41): ใช้ลูป while true เพื่อแสดงสถานะน้ำหนักเป้าหมาย และรับคำสั่งป้อนข้อมูลตัวอักษร (Z, J, K, C, L) ผ่านแป้นพิมพ์
3. ระบบปรับตั้งค่าศูนย์ / Tare (บรรทัดที่ 43–63): เมื่อป้อนคำสั่ง Z สคริปต์จะสั่งรัน Simulink แล้วดึงข้อมูล 50 จุดสุดท้าย (y_50_points) มาคำนวณหาค่าเฉลี่ยสะสม เพื่อบันทึกเป็นค่าแรงดันอ้างอิง ณ สถานะไม่มีน้ำหนัก (v_zero_offset)
4. การสุ่มเก็บข้อมูลและลดสัญญาณรบกวน (บรรทัดที่ 64–93): เมื่อป้อนคำสั่ง J สคริปต์จะสั่งรัน Simulink เพื่ออ่านข้อมูล 50 จุดสุดท้าย นำมาหาค่าเฉลี่ยแล้วหักล้างด้วย v_zero_offset เพื่อให้ได้ค่าแรงดันสุทธิ (avg_v) จากนั้นบันทึกค่าน้ำหนักและแรงดันสะสมลงในอาร์เรย์
5. การจัดการข้อมูล และการออกจากโปรแกรม (บรรทัดที่ 95–118): คำสั่ง K ใช้สำหรับลบข้อมูลการชั่งล่าสุด คำสั่ง C ใช้สำหรับล้างข้อมูลทั้งหมดเพื่อเริ่มวัดใหม่ และคำสั่ง L สำหรับจบการทำงานและเข้าสู่กระบวนการประมวลผลสรุป
6. การส่งออกข้อมูลเป็นไฟล์ Excel และสร้างกราฟ Calibration Curve (บรรทัดที่ 4–48 หน้า 2/2):
 - บันทึกชุดข้อมูลน้ำหนักและแรงดันที่ปรับลบ Zero แล้ว ลงในไฟล์ .xlsx โดยอัตโนมัติ (พร้อมระบบตรวจสอบโฟลเดอร์ save_path และตั้งชื่อไฟล์ต่อท้าย _1, _2 หากมีชื่อไฟล์ซ้ำ)
 - คำนวณหาสมการเชิงเส้น (Calibration Equation) ด้วยฟังก์ชัน polyfit() พร้อมพล็อตแสดงผลกราฟ Sensor Calibration Curve (Tare Applied) แสดงความสัมพันธ์ระหว่างน้ำหนักและแรงดันไฟฟ้าพร้อมเส้นสมการ

4.2 สคริปต์สำหรับการโหลดไฟล์ CSV และพล็อตแสดงผลกราฟ

สคริปต์นี้ถูกออกแบบมาเพื่อนำข้อมูลจากไฟล์ .csv มาประมวลผลคำนวณค่าน้ำหนักจริง และสร้างกราฟแสดงความสัมพันธ์เพื่อวิเคราะห์คุณลักษณะของเซนเซอร์ โดยมีขั้นตอนการทำงานดังนี้

```

1 % =====
2 % LAB 1.4: Single Point Load Cell with INA125 Data Plotting Script
3 % =====
4
5 % 1. อ่านข้อมูลจากไฟล์ CSV
6 data = readtable('Lab1.4.csv');
7
8 % 2. กำหนดค่าพารามิเตอร์และดึงข้อมูลจากตาราง
9
10 V_Q = -0.14725 ;
11 m = 0.33177;
12
13 Weight = (data.Weight_g)/1000;
14
15 Avg_1 = data.Adjusted_Voltage_V_1 ;
16 Avg_2 = data.Adjusted_Voltage_V_2 ;
17 Avg_3 = data.Adjusted_Voltage_V_3 ;
18 Avg = data.Adjusted_Voltage_V_AVERAGE ;
19
20 % =====
21 % กราฟเปรียบเทียบ Weight กับ แรงดันไฟฟ้า
22 % =====
23
24 fig1 = figure(1);
25 set(fig1, 'Name', 'Weight & Voltage', 'NumberTitle', 'off');
26
27 plot(Weight, Avg_1, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Trial
28 1');
29 hold on;
30 plot(Weight, Avg_2, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Trial
31 2');
32 plot(Weight, Avg_3, '-', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Trial
33 3');
34 plot(Weight, Avg, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName', 'Average
35 Voltage');
36 hold off;
37
38 title('กราฟแสดงความสัมพันธ์ระหว่างน้ำหนัก (Weight) และแรงดันไฟฟ้า (Voltage)');
39 ylabel('Weight (kg)');
40 xlabel('Voltage (V)');
41 legend('Location', 'best');
42 grid on;
43
44 % =====
45 % กราฟเปรียบเทียบ Weight_calculated กับ แรงดันไฟฟ้า
46 % =====
47
48 fig2 = figure(2);
49 set(fig2, 'Name', 'Weight_calculated & Voltage', 'NumberTitle', 'off');
50
51 plot(Avg, (Avg-V_Q)/m, '-o', 'LineWidth', 1.5, 'MarkerSize', 6, 'DisplayName',
52 'Average Voltage');
53 hold on;
54 hold off;
55
56 title('กราฟแสดงความสัมพันธ์ระหว่างน้ำหนักคำนวณ (Weight calculated) และแรงดันไฟฟ้า (Voltage)');
57 xlabel('Voltage (V)');
58 ylabel('Weight (kg)');
59 legend('Location', 'best');
60 grid on;

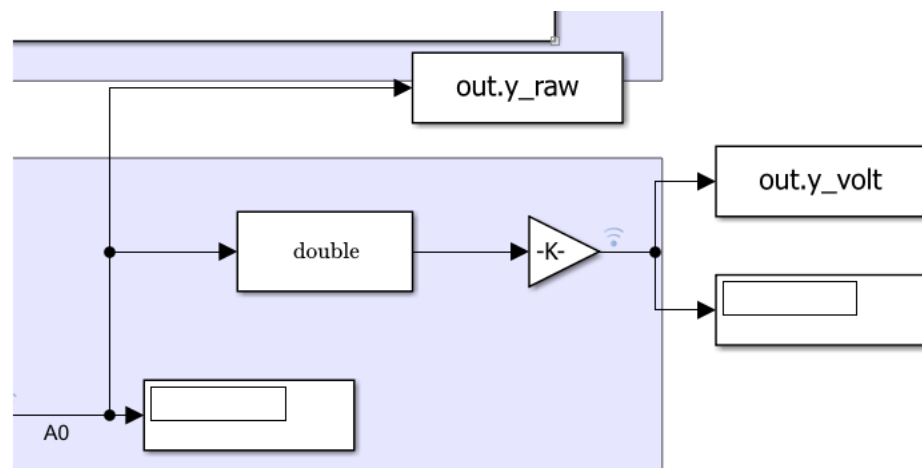
```

1. การโหลดข้อมูลจากไฟล์ (บรรทัดที่ 5-6): ใช้ฟังก์ชัน readtable() เพื่อดึงข้อมูลจากไฟล์ Lab1.4.csv เข้าสู่ตัวแปร data
2. การกำหนดค่าพารามิเตอร์และการแปลงหน่วย (บรรทัดที่ 8-18):
 - กำหนดค่าคงที่จากสมการ Calibration ได้แก่ ค่าแรงดัน ณ จุดศูนย์ $V_Q = -0.14725V$ และค่าความชัน $m = 0.33177V/kg$
 - แปลงหน่วยน้ำหนักจากกรัม (g) เป็นกิโลกรัม (kg) โดยหารด้วย 1000 เก็บไว้ในตัวแปร Weight
 - ดึงข้อมูลแรงดันไฟฟ้าจากการทดลองทั้ง 3 รอบ (Avg_1, Avg_2, Avg_3) และค่าแรงดันไฟฟ้าเฉลี่ย (Avg) ออกมาจากตารางข้อมูล
3. การพล็อต กราฟรูปที่ 1: ความสัมพันธ์ระหว่างน้ำหนักและแรงดันไฟฟ้า (บรรทัดที่ 24-38):
 - สร้างหน้าต่างแสดงผล figure(1) ชื่อ 'Weight & Voltage'
 - พล็อตเส้นเปรียบเทียบค่าแรงดันของการทดสอบ Trial 1, Trial 2, Trial 3 และค่าแรงดันเฉลี่ย (Average Voltage) สัมพันธ์กับค่าน้ำหนัก (\$kg\$)
 - กำหนดชื่อกราฟ ชื่อแกน (X: Voltage (V), Y: Weight (kg)) แสดง Legend และเปิดใช้งานเส้น Grid

4. การพล็อต กราฟรูปที่ 2: ความสัมพันธ์ระหว่างน้ำหนักคำนวณและแรงดันไฟฟ้า (บรรทัดที่ 44-55):

- สร้างหน้าต่างแสดงผล figure(2) ชื่อ 'Weight_calculated & Voltage'
- คำนวณน้ำหนักย้อนกลับจากค่าแรงดันไฟฟ้าด้วยสมการ $Weight_{calculated} = \frac{V_{out}-V_Q}{m}$ และพล็อตเปรียบเทียบกับค่าแรงดันไฟฟ้าเฉลี่ย (Avg)
กำหนดชื่อกราฟ ชื่อแกน (X: Voltage (V), Y: Weight (kg)) แสดง Legend และเปิดใช้งานเส้น Grid

4.3 แบบจำลอง Simulink



แบบจำลอง Simulink (Loadcell หรือ Simulink_Lab1.4) ถูกออกแบบมาเพื่อรับสัญญาณอนาล็อกจากเซนเซอร์ผ่านบอร์ดไมโครคอนโทรลเลอร์ โดยมีการประมวลผลภายในโมเดลดังนี้

- 4.3.1. บล็อก To Workspace (out.y_raw): ส่งออกข้อมูลสัญญาณดิจิทัลดิบ (Raw ADC Bits) เข้าสู่ MATLAB Workspace เพื่อใช้สังเกตหรือบันทึกค่าดิบ
- 4.3.2. บล็อก Data Type Conversion (double): ทำหน้าที่แปลงประเภทข้อมูลจากแบบ Integer/Unsigned ให้เป็นแบบ double เพื่อรองรับการคำนวณค่าทศนิยมที่มีความละเอียดสูง
- 4.3.3. บล็อก Gain: รับค่า ADC ดิจิทัล (0-4095) แล้วคูณด้วยปัจจัยขยาย ($\frac{3.3}{4095}$) เพื่อแปลงให้อยู่ในหน่วยแรงดันไฟฟ้า Volts (0-3.3V)
- 4.3.4. บล็อก To Workspace (out.y_volt): ส่งออกข้อมูลค่าแรงดันไฟฟ้าที่แปลงเรียบร้อยแล้วเข้าสู่ MATLAB Workspace ให้สคริปต์ LoadCell_Calibration.m นำไปหาค่าเฉลี่ย (Sample Averaging 50 จุด) และประมวลผลเซตศูนย์ (Tare / Set Zero) ต่อไป